INTRODUCCIÓN

K9 CALM

MANUAL COMPLETO / CONSULTA TÉCNICA

Pechera inteligente con ESP32-C3, sensores y Bluetooth. Para comenzar con menos teoria, utilizar la Guía Facil del mismo repositorio.



Ilustración conceptual generada con IA sobre un maniquí; no es fotografía de un prototipo probado ni un esquema eléctrico.

Descarga los codigos y las bibliotecas

https://github.com/f-lopez-velazquez/K9-CALM

Hecho por el profesor Francisco López Velázquez.
zolvek.com.mx

> **Alcance.** Cinco prácticas acumulativas. Pruebas en mesa o peluche, sin perros. La detección experimental no diagnóstica ni trata miedo.

Edición 3.0 · Identidad editorial Zolvek · Tamaño carta.

INTRODUCCIÓN

Antes de conectar

Construiremos un registrador de eventos. Un micrófono transforma cambios de presión del aire en muestras digitales. Un sensor inercial mide aceleración y giro. El ESP32-C3 busca un impulso acústico, compara actividad antes y después y comunica una categoría experimental.



Figura 1. Ambas entradas llegan al microcontrolador. El teléfono consulta el resultado por BLE.

Ruta corta	Que se obtiene	Pag.
Introducción	Alcance y reglas de laboratorio	1
Sección I · Hardware	Materiales, placa, sensores y etapas H1-H5	3
Práctica 1	Configurar y controlar	23
Práctica 2	Medir el movimiento	27
Práctica 3	Escuchar y analizar	32
Práctica 4	Batería y Bluetooth	40
Práctica 5	Integrar el detector	49
Cierre	Pruebas, diagnóstico, glosario y referencias	59

Al concluir: medir movimiento, caracterizar sonidos, leer voltaje, cambiar modo y consultar eventos en Android. La clasificación admite actividad normal, evento acústico sospechoso, posible reacción de sobresalto e información insuficiente.

Información. La compilación comprueba sintaxis y enlaces. La revisión teórica comprueba coherencia eléctrica. Ninguna sustituye medir la placa real. El registro de verificación al final distingue estos niveles.

SECCIÓN I / HARDWARE

Comprar con criterio

La lista completa CSV incluye cantidades, especificación, función, búsqueda en Mercado Libre y práctica. Las siguientes tablas permiten comprar sin abrir archivos externos. Los importes son **reservas de planeación no cotizadas**, no precios actuales verificados; no incluyen envio. Solicitar cotización antes de comprar.

Material de laboratorio por equipo

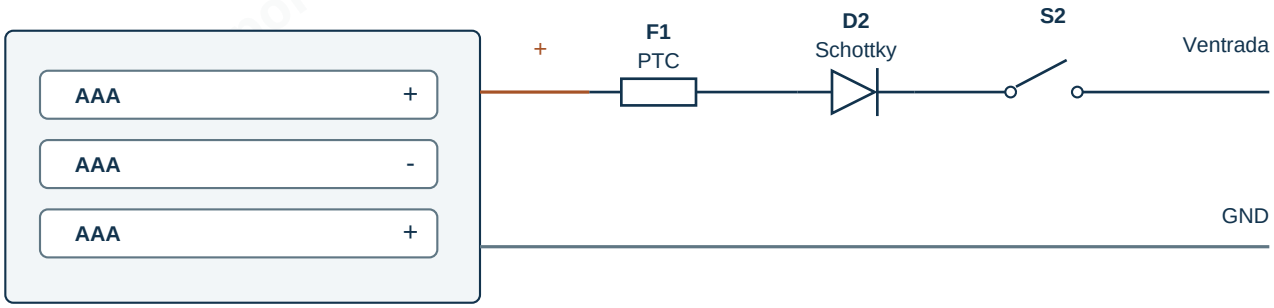
Componente	N	Especificación corta	MXN/u
ESP32-C3 SuperMini	1	USB-C; 16 terminales; ESP32-C3 con flash 4 MB	100
MPU6050 GY-521	1	Módulo con regulador y pull-ups a 3.3 V verificables	65
INMP441	1	Micrófono I2S; VDD 3.3 V; L/R accesible	100
LED rojo	1	3 mm; Vf aproximada 1.8-2.2 V	3
Resistencia LED	1	330 ohm; 1/4 W; 5%	1
Pulsador	1	Normalmente abierto; 2 o 4 patas	5
Protoboard	1	400 o 830 puntos; revisar separacion de placa	70
Cables Dupont	1	Juego M-M y M-H; cortos para I2S	65
Cable USB-C	1	USB datos y alimentación, no solo carga	70
Cabeceras	1	2 tiras 1x8 y tiras para sensores; paso 2.54 mm	12
Pulldown SD	1	100 kohm; 1/4 W; 1%	1
Desacoplos sensores	2	100 nF X7R en VDD-GND si módulo no los incorpora	2

SECCIÓN I / HARDWARE

Completar la modalidad portátil

Solo para modalidad portátil

Componente	N	Especificación corta	MXN/u
Pilas AAA	3	Alcalinas 1.5 V; mismo lote; no recargables en este diseno	18
Portapilas	1	3 AAA en serie; tapa y cables con polaridad	35
Interruptor	1	SPST; >=0.5 A a 6 V CC; preferir desconexión física del pack	10
PTC F1	1	Ihold 0.5 A orientativo; validar Itrip, temperatura y curva	8
Diodo D1	1	Schottky SS14 o 1N5819; 1 A; respetar banda	4
Resistencia R1	1	150 kohm; 1%; arriba del divisor	1
Resistencia R2	1	100 kohm; 1%; abajo del divisor	1
Capacitor ADC	1	100 nF cerámico X7R; >=10 V	2
Capacitor entrada	1	100 uF electrolítico; 10 V; observar polaridad	5
Conector polarizado	1	2 vias, desmontable; corriente >=0.5 A	12
Placa perforada	1	Fibra; pads aislados; tamaño según caja	30
Caja y fijaciones	1	Aislante; sin bordes; tornillos, separadores, velcro	80
Consumibles	1	Cable flexible, termocontráctil, soldadura y flux	50



B1 / 3 celdas en serie

Conector polarizado + tapa + aislamiento

Representación de conexiones. No recargar pilas alcalinas ni conectar USB y pack a la vez.

Portapilas e interruptor: las tres celdas internas están en serie; el dibujo no es un plano para fabricar el portapilas.

SECCIÓN I / HARDWARE

Herramientas y presupuesto

Una estación compartida

Elemento	N	Especificación	MXN/u
Multímetro	1	CC voltaje/corriente; fusible en entrada de corriente	250
Fuente de banco	1	0-6 V; límite de corriente ajustable; verificar salida	1200
Cautín y soporte	1	Temperatura regulada; punta fina	400
Herramientas y protección	1	Pinzas, pelacables, lentes; extractor de humo disponible	300
Peluche y soporte	1	Sin animales; base para movimientos controlados	100

Equipo existente

Elemento	N	Especificación	MXN/u
Computadora	1	USB y Arduino IDE 2.x; acceso a internet para instalar	0
Teléfono Android	1	BLE y nRF Connect; permisos de dispositivos cercanos	0

Extensiones separadas

Elemento	N	Especificación	MXN/u
Motor y etapa	1	Motor 3 V + NMOS logic level + diodo + R100 + R100k + C100nF/100uF	65
Fuente motor	1	3.0 V regulados; capacidad superior a corriente de arranque	90
Pull-ups I2C	2	4.7 kohm; solo si faltan o se retiran las de 5 V	1

Escenario	1 equipo	10 equipos
Solo material obligatorio	$496	$4960
Obligatorio + portátil	$788	$7880
Con una estación compartida	$3038	$10130
Reserva adicional sugerida 15%	$456	$1520

El material opcional suma $157 por equipo y no esta incluido en los totales. Computadora y teléfono se consideran existentes. Diez equipos comparten una estación: planear turnos para soldadura y mediciones; ampliar instrumentos si se requiere trabajo simultáneo.

SECCIÓN I / HARDWARE

Lista de compra trazable

Búsqueda literal sugerida en Mercado Libre y momento de uso. No son enlaces ni existencias verificadas. Función y especificación se encuentran en las tablas previas y en el CSV.

Pieza	Búsqueda sugerida	P
ESP32-C3 SuperMini	ESP32 C3 SuperMini USB C 4MB	1
MPU6050 GY-521	GY 521 MPU6050 acelerómetro giroscopio	2
INMP441	INMP441 I2S micrófono digital	3
LED rojo	LED rojo 3mm difuso	1
Resistencia LED	resistencia 330 ohms cuarto watt	1
Pulsador	pulsador tactil 6x6 normalmente abierto	1
Protoboard	protoboard 830 puntos	1
Cables Dupont	cables dupont macho hembra macho macho	1
Cable USB-C	cable USB A USB C datos	1
Cabeceras	pines header macho 2.54 mm	1
Pulldown SD	resistencia 100k 1 porciento	3
Pilas AAA	pilas AAA alcalinas	4
Portapilas	portapilas 3 AAA con tapa	4
Interruptor	interruptor mini SPST 6V	4
PTC F1	fusible rearmable PTC 0.5A	4
Diodo D1	diodo 1N5819 Schottky	4
Resistencia R1	resistencia 150k 1 porciento	4
Resistencia R2	resistencia 100k 1 porciento	4

SECCIÓN I / HARDWARE

Completar la lista de compra

Pieza	Búsqueda sugerida	P
Capacitor ADC	capacitor 100nf cerámico	4
Capacitor entrada	capacitor 100uF 10V	4
Desacoplos sensores	capacitor 100nf X7R	3
Conector polarizado	conector polarizado 2 pines cable	4
Placa perforada	placa perforada universal fibra	5
Caja y fijaciones	caja plastica electrónica pequeña velcro	5
Consumibles	kit termocontráctil cable flexible soldadura	5
Multímetro	multímetro digital fusible corriente	1
Fuente de banco	fuente laboratorio regulable límite corriente	4
Cautín y soporte	estación soldadura regulable	1
Herramientas y protección	kit herramientas electrónica lentes	1
Peluche y soporte	peluche perro	2
Computadora	equipo disponible no presupuestado	1
Teléfono Android	teléfono disponible no presupuestado	4
Motor y etapa	motor vibrador 3V MOSFET logic level	5
Fuente motor	fuente regulada 3V motor	5
Pull-ups I2C	resistencia 4.7k	2

Inspeccion de recepcion

Separar piezas y comprobar valores con el multímetro antes de soldar. El módulo GY-521 debe poder trabajar a 3.3 V sin elevar SDA/SCL a 5 V. El INMP441 debe exponer L/R. Solicitar al vendedor el esquema y referencia del regulador de la SuperMini; la apariencia no basta para identificarlo.

> **Precaución.** Si las cabeceras vienen sin soldar, usar cautín con soporte, ventilación y lentes. Trabajar sin energía. No sujetar una placa conectada con pinzas metalicas.

SECCIÓN I / HARDWARE

La placa que usaremos

Un microcontrolador es una computadora pequeña: ejecuta instrucciones, lee entradas y controla salidas. El ESP32-C3 utiliza un núcleo RISC-V y ofrece BLE. La placa SuperMini agrega USB-C, regulador y botones para que podamos programarlo. “SuperMini” describe una familia de placas de terceros, no un modelo único de Espressif.

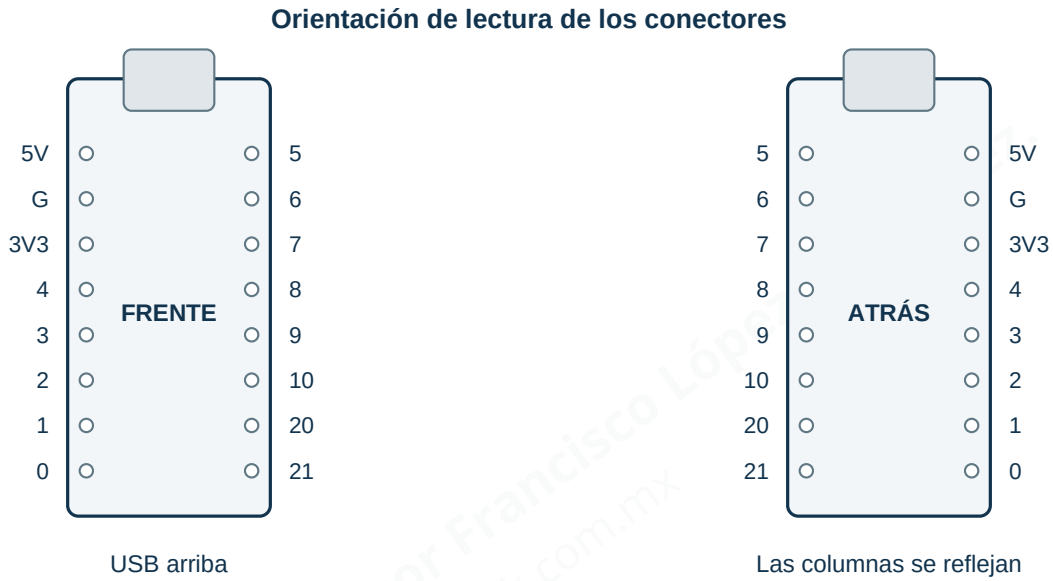


Figura 2. Reconstrucción esquematizada de la variante común de 16 terminales; el reverso refleja las columnas. No es una fotografía ni certifica la unidad comprada.

Referencia geométrica: USB arriba; columna izquierda 5V, GND, 3V3, 4, 3, 2, 1, 0; derecha 5, 6, 7, 8, 9, 10, 20, 21. Si no coincide, detener el cableado y elaborar la correspondencia con el esquema del proveedor. El LDO no se da por identificado: registrar su marcaje y ficha antes de H4.

GPIO, USB y límites

GPIO significa entrada/salida de propósito general. Un pin puede leer un botón o producir un nivel lógico, pero no alimentar motores. Los GPIO trabajan a 3.3 V; nunca aplicarles 5 V. El USB nativo utiliza GPIO18/19. Los pines 2, 8 y 9 participan en el arranque: quedan libres en este curso. GPIO4/5 se reasignan a I2C; no se usa JTAG externo.

SECCIÓN I / HARDWARE

Mapa de pines definitivo

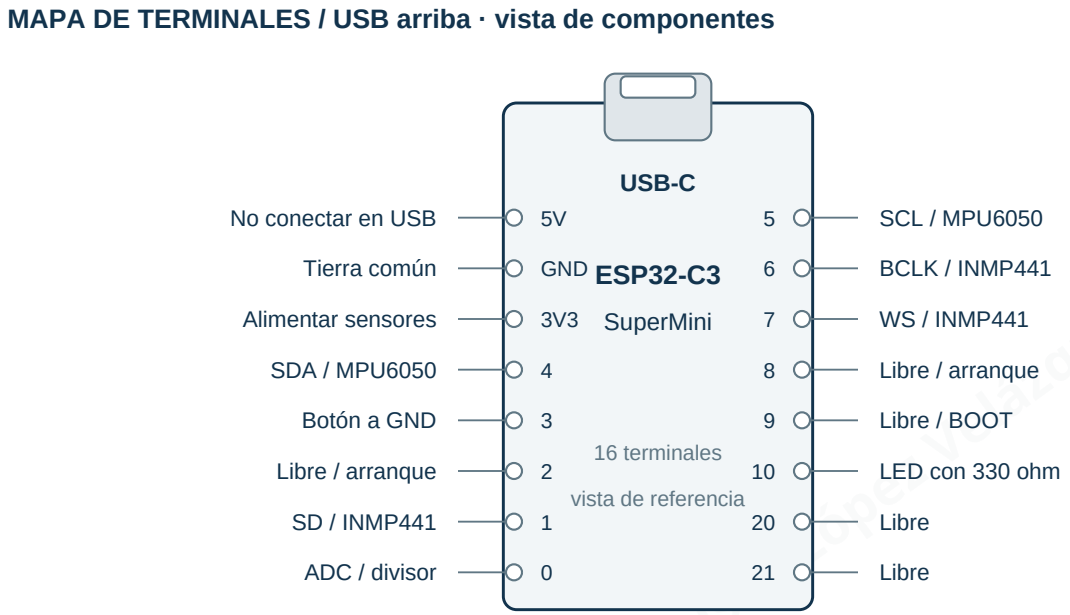


Figura 3. Pinout de la variante de referencia. El nombre del GPIO en código corresponde al número serigrafiado, no a la posición física.

No conectar por posición de memoria: leer cada etiqueta con la placa orientada como el dibujo. USB nativo usa GPIO18/19 internamente; no confundir los números 20 y 21 con USB. La referencia es el esquema de distribuidor R13 y la ficha C3 R1; ambas deben coincidir con la unidad comprada.

Hecho por el profesor Francisco López Velázquez.

SECCIÓN I / HARDWARE

Tabla maestra de conexiones

Función	ESP32-C3	Destino y condicion
LED externo	GPIO10	330 ohm -> ánodo; cátodo -> GND
Botón	GPIO3	Pulsador -> GND; INPUT_PULLUP
I2C SDA	GPIO4	GY-521 SDA; pull-up solo a 3V3
I2C SCL	GPIO5	GY-521 SCL; pull-up solo a 3V3
I2S BCLK	GPIO6	INMP441 SCK
I2S WS	GPIO7	INMP441 WS
I2S entrada	GPIO1	INMP441 SD; 100k a GND
ADC batería	GPIO0	Nodo R1=150k / R2=100k
Sensores	3V3 / GND	VCC o VDD / GND; AD0 y L/R a GND
Sin utilizar	2, 8, 9	Strapping; BOOT=9, LED común=8
Reservados	20, 21	UART; 20 solo extensión opcional
USB nativo	18, 19	D-/D+; no son puntos de conexión del curso

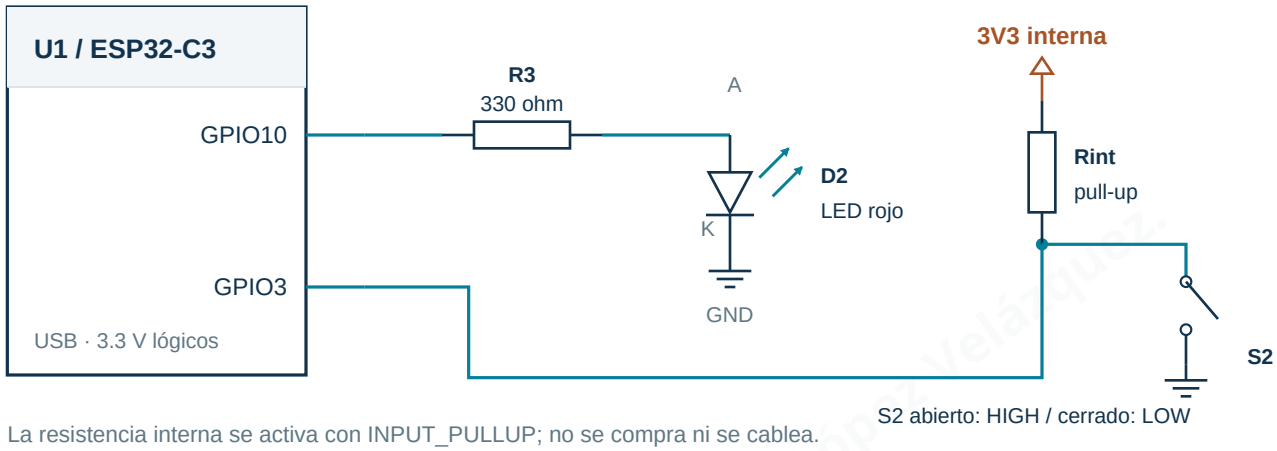
Una red, varios terminales

Una red eléctrica une puntos al mismo potencial. GND conecta negativos y referencias; 3V3 alimenta sensores. Compartir GND no significa unir 3V3 con 5V. Las líneas de color representan redes diferentes y siempre llevan etiqueta: el color por si solo no define la conexión. Las resistencias y capacitores de H4 no se omiten al pasar al montaje compacto.

> **Información.** Usamos LED externo en GPIO10 para que el curso no dependa del LED integrado, frecuentemente activo en LOW en GPIO8. Verificar la propia placa antes de usar ese LED.

SECCIÓN I / HARDWARE

H1 · Luz y un botón



La resistencia interna se activa con INPUT_PULLUP; no se compra ni se cablea.

Figura 4. Dos circuitos sencillos con tierra común. El botón se mide; el LED se controla.

Origen	Trayecto	Destino
GPIO10	Resistencia 330 ohm -> ánodo LED	Cátodo LED -> GND
GPIO3	Pulsador normalmente abierto	GND

Por qué funciona

El LED conduce principalmente en un sentido. La pata larga suele ser ánodo y el lado plano indica cátodo: confirmar con prueba de diodo. Una resistencia limita la corriente: I=(3.3-2.0)/330=0.00394 A, aproximadamente 4 mA. Sin ella, el LED o el GPIO pueden dañarse. La resistencia puede estar antes o después del LED porque ambos están en serie.

INPUT_PULLUP activa una resistencia interna que mantiene el botón en HIGH cuando está abierto. Al presionarlo, conecta la entrada a GND y se lee LOW. Un pulsador de cuatro patas tiene pares unidos internamente: identificarlos con continuidad, sin energía. Si se conectan dos patas del mismo par, parecera presionado siempre.

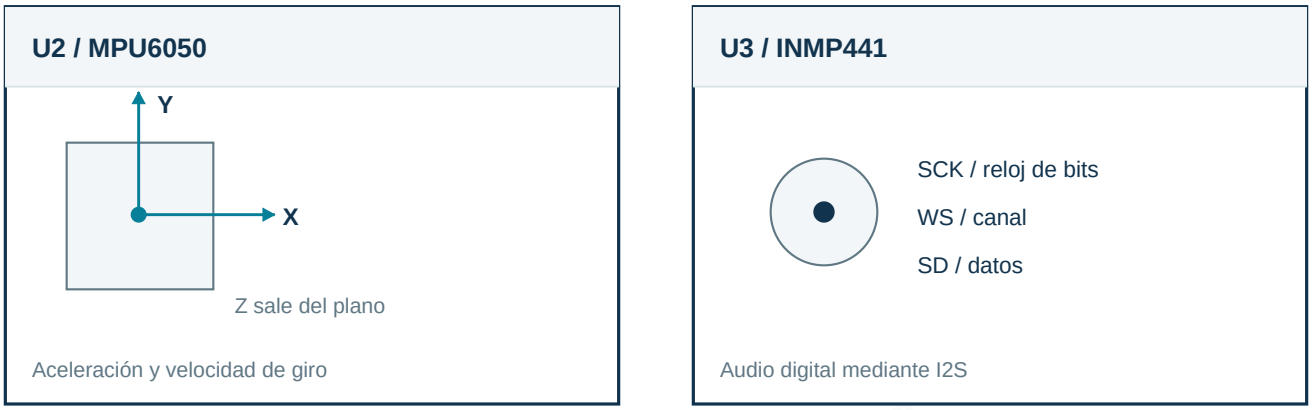
Comprobar antes de programar

Sin USB, buscar cortos entre 3V3 y GND. En modo voltaje CC y con USB, punta negra en GND: 3V3 debe estar cerca de 3.3 V. Con P1, GPIO3 pasa de cerca de 3.3 V a 0 V al presionar. Un GPIO de LED alternante puede mostrar un promedio en el multímetro.

> **Verificación.** LED con resistencia / Botón cambia al presionar / Ningun GPIO recibe 5 V

SECCIÓN I / HARDWARE

H2 · Entender el movimiento



Ejes y orden físico: coteja la serigrafía del módulo comprado.

Figura 5. Sensores esquematizados; comprobar ejes y orden de terminales en el módulo real.

El acelerómetro mide aceleración especifica en tres ejes. En reposo sobre la mesa registra aproximadamente 9.81 m/s² por el apoyo contra la gravedad; en caída libre ideal se acerca a cero. Inclinar la placa reparte la lectura entre ejes. El giroscopio mide velocidad angular: que tan rápido gira, no el ángulo absoluto. Adafruit entrega rad/s: 1 rad/s equivale a unos 57.3 grados/s.

Una medida sencilla de actividad

A = sqrt(ax² + ay² + az²), en m/s². Usamos **d = |A - 9.80665|** y un filtro **act = act + 0.15 × (d - act)**. Si act era 0 y d=2, el nuevo valor es 0.30 m/s². El filtro suaviza vibraciones y ruido, pero retrasa los cambios. Esta medida no elimina toda la gravedad ni detecta bien un giro suave: también se registra el giroscopio.

La frecuencia de muestreo de movimiento es 50 lecturas/s: una cada 20 ms. La frecuencia de un movimiento es otra cosa: dos oscilaciones por segundo son 2 Hz. Para estudiar regularidad, comparar intervalos entre picos; variaciones de montaje y golpes producen señales parecidas. No inferir conducta solo de la magnitud.

SECCIÓN I / HARDWARE

H2 · Conectar y verificar I2C

SDA y SCL necesitan pull-ups a 3V3; comprueba los del módulo.

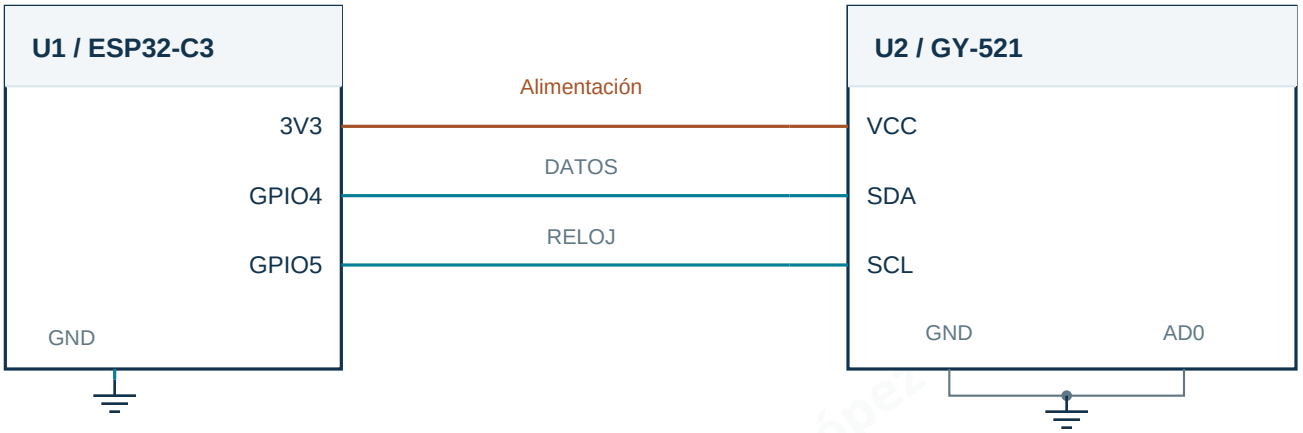


Figura 6. H1 permanece; se añade GY-521. INMP441 todavía no se monta.

GY-521	ESP32-C3	Motivo
VCC / GND	3V3 / GND	Energía compatible y referencia común
SDA / SCL	4 / 5	Datos y reloj definidos por Wire.begin(4,5)
AD0	GND	Dirección I2C 0x68
XDA, XCL, INT	Sin conectar	Funciones no utilizadas

I2C es una conversacion sobre dos líneas. SDA transporta datos; SCL marca el ritmo. El ESP32 inicia las consultas y el sensor responde a su dirección, como un número de casa. Ambas líneas requieren resistencias pull-up: los dispositivos bajan la línea y las resistencias la devuelven a 3.3 V. Muchas GY-521 ya las incluyen. No agregar más sin revisar el esquema.

Con USB y sin intercambio, medir SDA y SCL: deben reposar cerca de 3.3 V. Si están a 5 V, desconectar y corregir pull-ups. Si están a 0 V, revisar cortos o sensor averiado. El escáner integrado en P2 muestra 0x68. AD0 alto daria 0x69, pero nuestro código espera 0x68; no cambiarlo inadvertidamente.

Precaución. Medir 3V3 en el módulo y revisar el regulador GY-521: una caída excesiva al alimentarlo con 3.3 V puede impedir que el chip reciba su rango permitido. Verificar su esquema; no puentear un regulador sin identificarlo.

SECCIÓN I / HARDWARE

H3 · El sonido se convierte en números

El INMP441 es un micrófono MEMS digital: una membrana responde a cambios de presión y la electrónica interna produce muestras. No se conecta al ADC. I2S transmite el flujo con reloj de bits (SCK/BCLK), selección de ranura (WS) y datos (SD). Es distinto de I2C: SDA/SCL consultan registros; I2S entrega audio continuo.

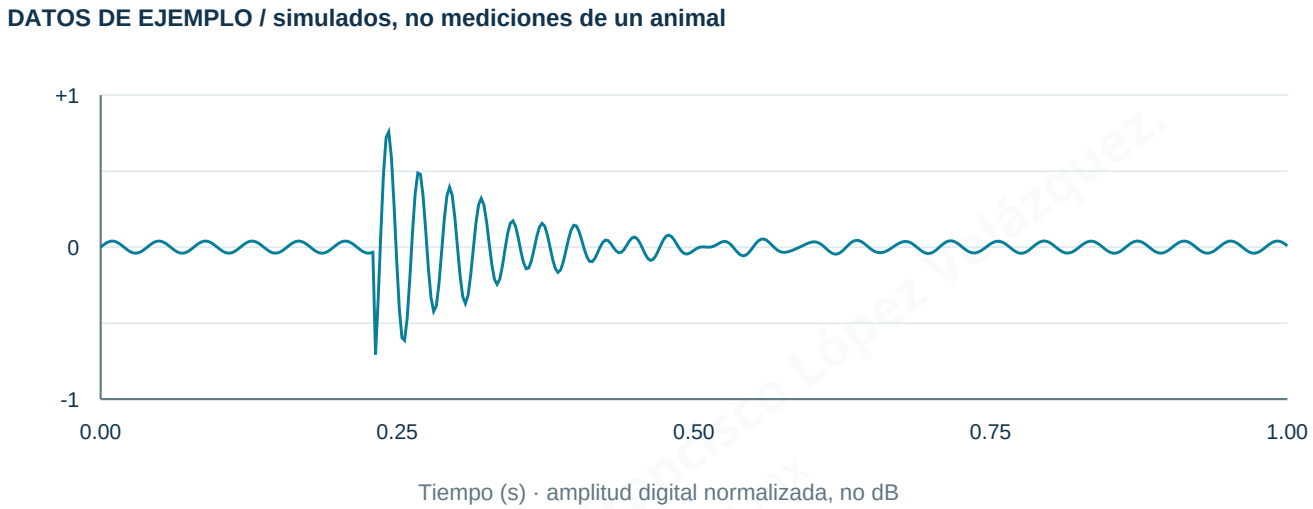


Figura 7. Señal impulsiva sintética, con ejes de tiempo y amplitud. No es un registro real ni identifica el origen.

Amplitud: tamaño de la muestra. **Pico**: mayor valor absoluto de una ventana. **RMS**: raiz de la media de los cuadrados, una medida de intensidad sostenida. **Duracion**: tiempo por encima del umbral. **Tono**: percepcion ligada a periodicidad; **frecuencia**: ciclos por segundo. Un aplauso tiene ataque rápido y decaimiento corto; un trueno puede durar más, pero reverberacion, distancia y grabacion alteran ambos.

Un portazo, una explosion de pirotecnia y un aplauso pueden generar impulsos de banda ancha. El prototipo solo marca eventos sospechosos. No reconoce con certeza su origen y no mide dB SPL sin calibración acústica. Los valores normalizados son relativos a escala digital, no a la salud auditiva.

SECCIÓN I / HARDWARE

H3 · Cablear I2S sin ambigüedad

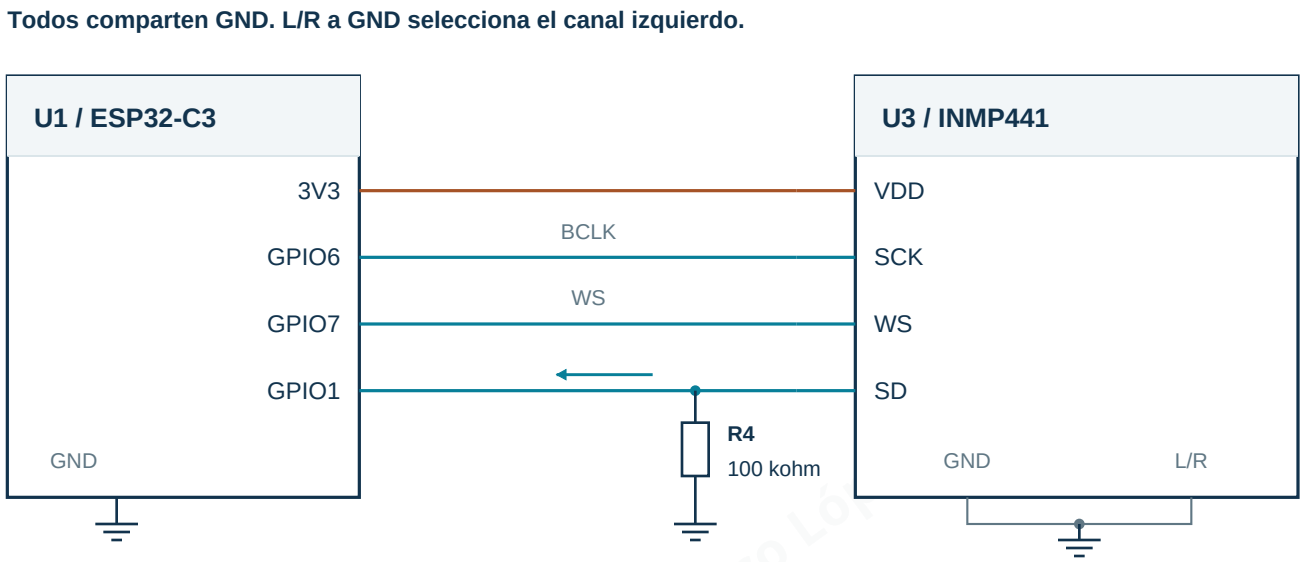


Figura 8. Todos los sensores simultáneos: I2C y I2S usan pines distintos.

INMP441	Destino	Comprobación
VDD / GND	3V3 / GND	Medir 3.3 V; no usar 5 V
SCK / WS	GPIO6 / GPIO7	Salida del C3: 1.024 MHz / 16 kHz
SD	GPIO1	Entrada de datos; 100k a GND
L/R	GND	Selecciona ranura izquierda
CHIPEN si expuesto	3V3	Revisar módulo; habilitacion en alto

Muestreamos a 16,000 Hz. Cada trama contiene dos ranuras de 32 bits: 16,000 × 64 = 1,024,000 pulsos de BCLK/s. El micrófono entrega 24 bits con signo en la ranura izquierda; el programa desplaza 8 bits y divide entre 8,388,608 para normalizar. La ranura derecha se descarta. No configurar una trama mono de 32 pulsos para este micrófono.

Usar cables cortos y capacitor de 100 nF junto a VDD-GND si falta. El orificio acústico no debe taparse. La hoja de datos recomienda 100k de SD a GND porque la salida queda en alta impedancia fuera de su ranura. Sin reloj, el multímetro no demuestra que I2S funcione: con P3 revisar que RMS no sea permanentemente cero ni cercano a 1; un osciloscopio permite verificar los relojes.

Verificación. L/R a tierra / No confundir SD con SDA / Ventanas de audio cambian al hablar suave

SECCIÓN I / HARDWARE

H4 · Elegir una sola fuente

Laboratorio: USB de datos para desarrollar H1-H3 y las cinco prácticas. El voltaje del portapilas se puede simular con una fuente aislada solo para medir el divisor, sin unir su positivo al pin 5V de la placa. **Portátil:** tres AAA en serie, sin USB, después de aprobar el ensayo del regulador. No se usa MT3608 ni litio.

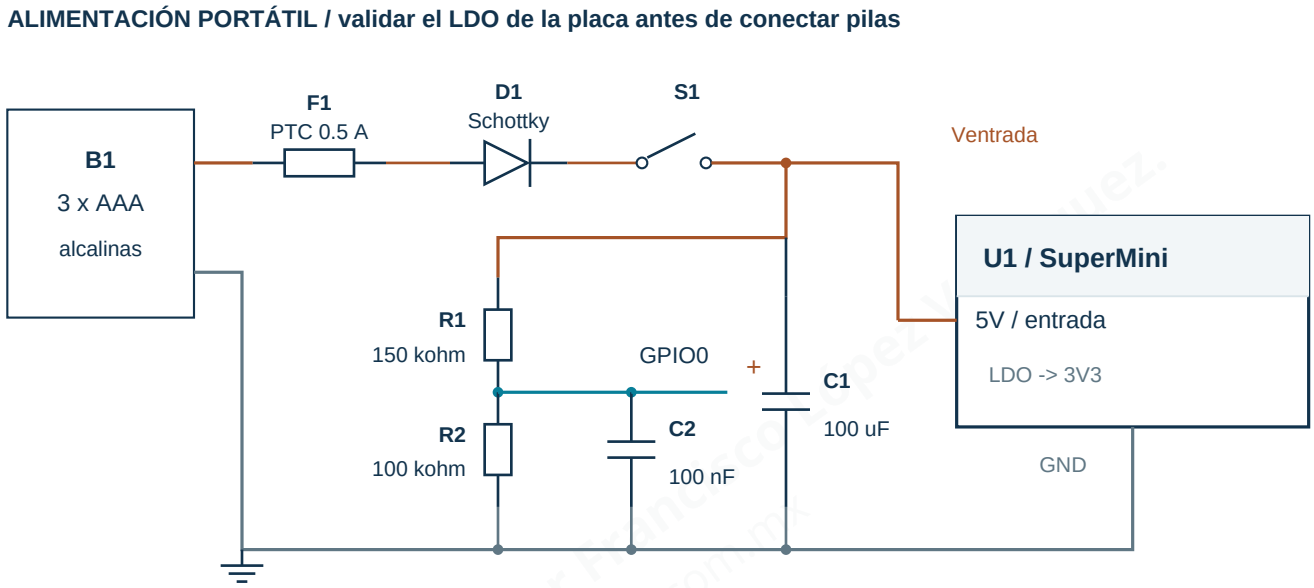


Figura 9. F1 y D1 reducen riesgos, pero añaden caída. R1 se toma después del interruptor para no inyectar corriente al ADC apagado.

Red	Conexión exacta
Positivo protegido	AAA+ -> F1 -> ánodo D1; cátodo D1 -> S1 -> nodo Ventrada
Entrada placa	Ventrada -> pin 5V/VBUS solo si esquema lo autoriza
Negativo	AAA- -> GND de placa, sensores, R2 y capacitores
Divisor	Ventrada -> R1 150k -> nodo GPIO0 -> R2 100k -> GND
Capacitores	100nF: GPIO0-GND; 100uF/10V: Ventrada-GND, + a Ventrada

Precaución. Nunca conectar 4.5 V a 3V3. D1 protege polaridad inversa, pero NO aísla el VBUS de USB. Retirar fisicamente el conector del pack antes de conectar USB; apagar S1 no debe ser la única barrera.

SECCIÓN I / HARDWARE

H4 · Validar el regulador real

Una pila alcalina nueva puede superar 1.5 V. Medir el conjunto; no asumir 4.5 V exactos. Un regulador lineal necesita una diferencia entre entrada y salida, llamada dropout, que depende de corriente y temperatura. Si la entrada cae demasiado, la salida deja de ser 3.3 V. El LED puede seguir encendido mientras el microcontrolador se reinicia.

Condicion: Vpack bajo carga - Vdiodo - VPTC - Vcables >= 3.3 V + Vdropout(I,T). Ejemplo solo aritmético: 4.0 - 0.25 - 0.05 = 3.70 V; quedaría margen de 0.40 V para el LDO. No es una especificación del regulador comprado.

Paso	Medición / criterio
1 · Identificar	Fotografiar marcaje LDO y obtener ficha: entrada máxima, salida, dropout, corriente, capacitores y térmica. Si es ilegible o no hay esquema, usar USB y no aprobar AAA.
2 · Continuidad	Sin fuentes: confirmar que 5V llega a entrada LDO; comprobar conexión a USB VBUS y ausencia de ORing. Confirmar polaridad y sin corto.
3 · Fuente limitada	Sin USB y sin pilas: alimentar entrada protegida con fuente ajustable. Limitar inicialmente a 100 mA para detectar cortos; aumentar solo con revisión de consumo y capacidad de LDO/cables, hasta permitir arranque.
4 · Barrido	Desde voltaje real del pack nuevo, reducir en pasos de 0.1 V con BLE conectado y ambos sensores. Medir entrada y 3V3 en reposo y eventos; registrar reinicios. Detener si cae de 3.20 V o excede ficha/temperatura.
5 · Umbrales	Tomar la menor entrada estable, añadir margen >=0.2 V y definir crítica; baja >=crítica+0.2 V. Sustituir constantes del código. Si requiere umbrales mayores que el pack disponible, rechazar la modalidad.
6 · Pilas	Retirar fuente; conectar AAA protegidas sin USB. Repetir pruebas. Una advertencia de software no desconecta la energía: apagar al aviso crítico.

Presupuestar picos de corriente, no solo el promedio. Para diseñar la fuente del ESP32-C3, Espressif recomienda capacidad de al menos 500 mA; verificar también la corriente disponible del LDO y sensores. No se promete autonomía: medir mA promedio y caídas con carga. Medir corriente EN SERIE, nunca poner el amperimetro entre + y GND.

SECCIÓN I / HARDWARE

H4 · Divisor, ADC e histéresis

El ADC convierte voltaje en un número. A 12 bits hay 4096 niveles, pero el resultado no equivale a una medición ideal. Usamos analogReadMilliVolts(), calibración del chip y una comparación adicional con multímetro. En C3, la atenuación mayor permite medir aproximadamente hasta 2.5 V; no suponer que todo el rango de 3.3 V es medible.

Vadc = Ventrada × R2/(R1+R2). Con 150k y 100k, la fraccion es 0.4; 4.50 V produce 1.80 V. Para reconstruir: **Ventrada = Vadc × 2.50 × CAL**. Se mide después de D1, por lo que no representa exactamente la tensión en bornes de las pilas. La corriente permanente del divisor a 4.5 V es 4.5/250000 = 18 microamperios.

Par arriba/abajo	4.5 V entrada	5.25 V entrada	Factor
120k / 100k (ejemplo)	2.045 V	2.386 V	2.20
150k / 100k (definitivo)	1.800 V	2.100 V	2.50

120k/100k puede servir tras comprobar tensión máxima y tolerancias; deja menos margen. Elegimos 150k/100k al 1%. Peor caso a 5.25 V, R1 baja 1% y R2 sube 1%: 5.25 × 101/(148.5+101)=2.125 V. El máximo de alimentación permitido por la placa se valida aparte: este calculo no autoriza alimentarla a 5.25 V.

La resistencia equivalente del divisor es 60k. Con 100nF, τ=RC=0.006 s; esperar al menos 30 ms tras energizar y promediar. El programa espera 1 s y promedia 16 lecturas. El capacitor suministra los pequeños pulsos del muestreo y reduce ruido. Confirmar estabilidad con multímetro y varios voltajes.

Calibración: si el multímetro indica 4.20 V y el programa 4.10 V, CAL=4.20/4.10=1.0244. Repetir en al menos tres puntos; una constante no corrige toda no linealidad. Si el error supera 0.10 V tras calibrar, investigar antes de usar alarmas.

Histéresis: entrar a baja por debajo de 4.00 V y salir solo sobre 4.15 V evita parpadeos por ruido. 3.80 V crítico y 4.00 V baja son ejemplos didacticos, desactivados por PILAS=false. El porcentaje de carga no se calcula: voltaje, carga, temperatura y edad no dan una relación universal.

SECCIÓN I / HARDWARE

H5 · Del cableado a la maqueta

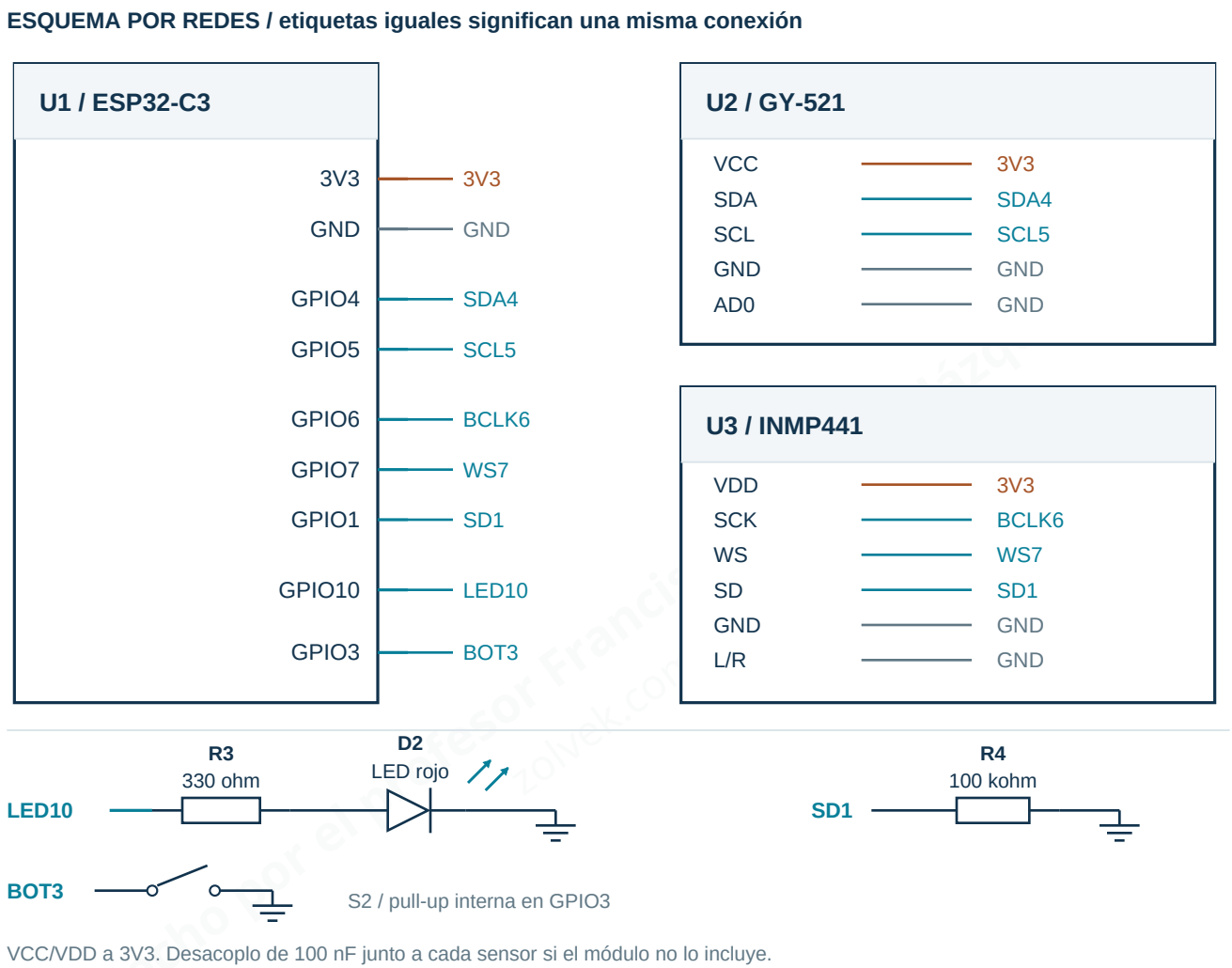


Figura 10. Circuito completo de laboratorio: USB como única alimentación; divisor solo si se ensaya H4 con entrada aislada.

Conexión integrada	Terminales
Alerta / control	10 -> 330 ohm -> LED -> GND; 3 -> botón -> GND
MPU6050	4 SDA, 5 SCL, 3V3 VCC, GND GND/AD0
INMP441	6 SCK, 7 WS, 1 SD +100k a GND; 3V3 VDD, GND GND/LR
Medida opcional H4	0 al divisor 150k/100k; 100nF nodo-GND
Energía	USB; pack retirado. AAA solo con autorización técnica de H4

SECCIÓN I / HARDWARE

H5 · Revisar el montaje

DISTRIBUCIÓN MECÁNICA / propuesta sobre una maqueta



Caja aislante, alivio de tensión y antena libre. No es una colocación terapéutica.

Distribucion mecánica para una maqueta: separar energía y sensores, fijar el MPU6050 y dejar libre el puerto acústico.

Probar primero en protoboard. Si la SuperMini tapa los agujeros vecinos, usar dos protoboards o extensiones; no doblar patas para forzarla. Pasar a placa perforada solo después de comprobar cada red por continuidad. Mantener cables I2S cortos, GND firme y sensores separados de cables del motor. El MPU debe quedar sujeto para que mida el objeto y no un módulo colgando.

Usar caja aislante, alivio de tensión y conectores polarizados. Dejar la antena libre de metal y las AAA separadas de puntos calientes. No sellar con pegamento el micrófono. La propuesta de pechera solo se monta en una maqueta o peluche: peso, temperatura, humedad, bordes, desprendimientos y bienestar animal requieren validación independiente.

Hecho por el profesor Francisco López Velázquez.

SECCIÓN I / HARDWARE

Extensión opcional: vibración

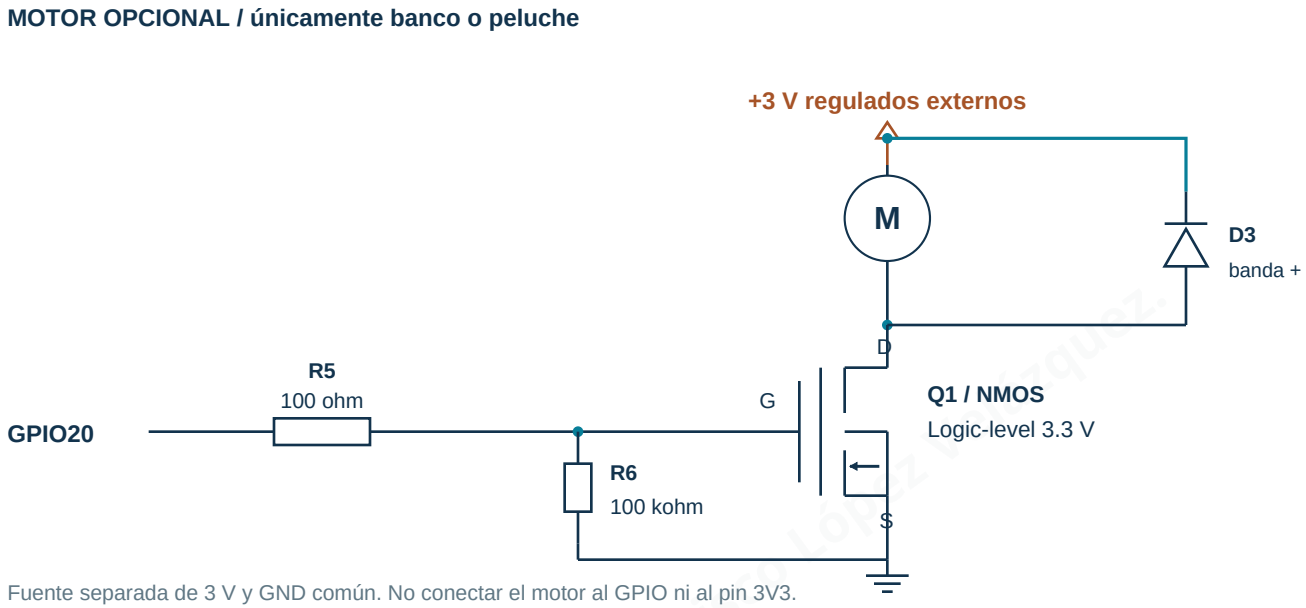


Figura 11. Etapa conceptual low-side con diodo de rueda libre. No está conectada ni habilitada en los cinco programas.

Elemento	Conexión / selección
Motor	+ a fuente 3.0 V regulada; - a drain NMOS
NMOS	Source a GND; gate por 100 ohm a GPIO20; 100k gate-GND
Diodo	Cátodo/banda a +3 V; ánodo a drain; corriente y pulso compatibles con motor
Fuente	3.0 V regulados externos; GND común con C3; capacidad mayor al arranque
Filtrado	100nF y 100uF junto al motor/fuente; electrolítico con polaridad

El GPIO solo manda la compuerta; la corriente del motor circula por transistor y fuente. Elegir NMOS cuya resistencia RDS(on) este especificada a 2.5 o 3.3 V, no solo por su voltaje de umbral. Comprobar patillaje en su hoja de datos. Al apagar una bobina aparece un pico inverso; el diodo le ofrece un camino y protege el transistor. Un diodo al reves cortocircuita la fuente al encender.

No alimentar el motor desde GPIO, 3V3 de la placa ni directamente con tres AAA: podría recibir más de 3 V y provocar reinicios. Probar por separado con fuente limitada, medir corriente de arranque y temperatura. Solo entonces integrar una temporización corta sin delay(). Prueba exclusivamente en banco o peluche; no implica que vibrar calme a un animal.

SECCIÓN I / HARDWARE

Vibración: qué respalda la ciencia

Alcance. No se identificó evidencia clínica suficiente para recomendar vibración en pecho, espalda o ambos para calmar perros. No se asigna una dosis por raza o kg ni se activa un motor sobre animales.

Fuente	Resultado útil / límite
IEEE WHC 2019 · DOI 10.1109/WHC.2019.8816079	Un perro aprendio ordenes con cuatro motores. Comunicación aprendida no equivale a reducción de miedo.
Animals 2026 · DOI 10.3390/ani16020323	Revisión de comunicación háptica: diferencias en protocolos y necesidad de medir percepcion y bienestar. No establece sitio calmante.
Pekkin 2016 · DOI 10.1016/j.applanim.2016.09.003	28 perros reclutados; chalecos de presión, no motores. Sin efecto terapéutico claro en el ensayo.
Animals 2024 · DOI 10.3390/ani14233445	Revisión sistematica de cuatro estudios de presión: evidencia debil y limitada, con sesgos e inconsistencia.

Implicación para el diseño

Se conserva LED/BLE como salida del detector. El motor es una extensión de banco, no un tratamiento. El dibujo de la caja dorsal muestra alojamiento de electrónica, no una ubicacion terapéutica. No se cambia al pecho sin evidencia y validación profesional.

Inferencia de ingenieria: pelo, ajuste, masa y montaje pueden cambiar la vibración transmitida. Tres voltios describen alimentación eléctrica, no aceleración en piel. PWM en un motor ERM no fija de forma independiente frecuencia mecánica e intensidad. La vibración también puede contaminar MPU6050 y micrófono.

No se identificó una pauta validada por raza, tamaño, intensidad, Hz o duracion para el objetivo de calmar. Tampoco se extrapolan ensayos humanos de chalecos vibratorios. Menor actividad no demuestra calma: se requieren medidas de bienestar y evaluacion veterinaria/etológica.

Informe y enlaces: Guía/Evidencia_vibracion_y_bienestar.md. Búsqueda dirigida; no es una revisión sistematica nueva.

SECCIÓN II / SOFTWARE · CINCO PRÁCTICAS

01 / Configurar y controlar

Objetivo. Instalar el entorno, subir un programa y cambiar entre automático y manual.

Lo que aprenderemos

Compilar frente a subir, puerto USB, niveles digitales, antirrebote y temporización con millis().

Materiales y punto de partida

ESP32-C3, cable de datos, LED, 330 ohm, botón y multímetro. No conectar sensores aun.

Comenzar con la placa desconectada y construir H1.

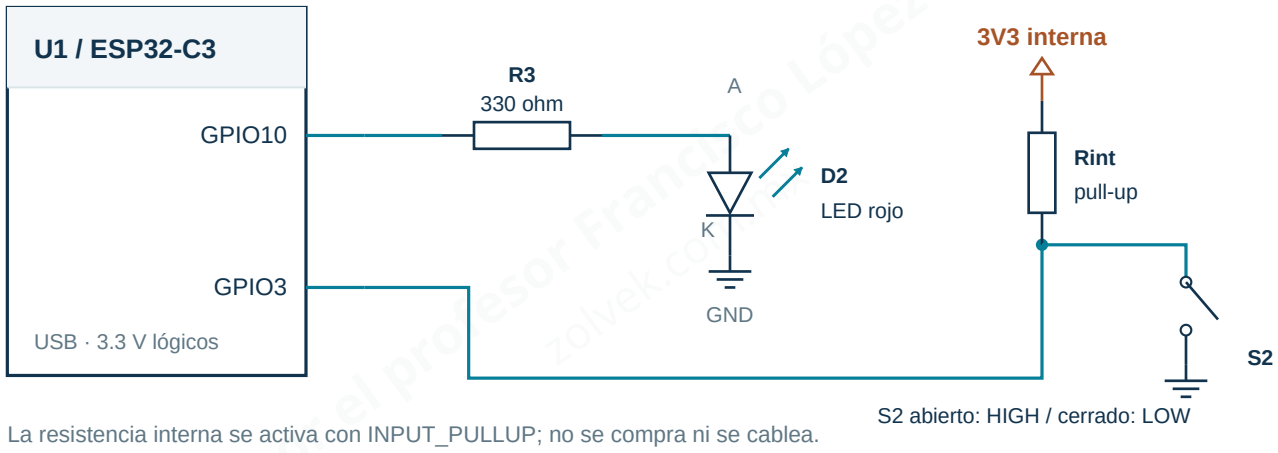


Figura P1. Mismas redes que en hardware; esquema funcional, no huella física.

Red	Conexión
LED	10 -> 330 ohm -> LED -> GND
Botón	3 -> pulsador -> GND

SECCIÓN II / SOFTWARE · CINCO PRÁCTICAS

P1 · Preparar Arduino IDE

1. Descargar Arduino IDE 2.x desde arduino.cc/en/software e instalar la versión de su sistema. Abrir Archivo > Preferencias (o Settings). En “URLs adicionales del Gestor de tarjetas” agregar la dirección oficial siguiente, sin espacios ni saltos:

https://espressif.github.io/arduino-esp32/package_esp32_index.json

2. Abrir Gestor de placas, buscar “esp32 by Espressif Systems” e instalar **3.3.4**, la versión fijada para este manual. No confundir con un paquete de otro fabricante. La versión más nueva puede cambiar APIs; si se actualiza, repetir compilaciones y validación de I2S y BLE.

3. Conectar por USB sin pilas. Seleccionar **ESP32C3 Dev Module** y el puerto que aparece al conectar. En Herramientas elegir **USB CDC On Boot: Enabled**, flash 4 MB y velocidad de subida 115200 si hay problemas. En placas con USB nativo el puerto puede reaparecer con otro nombre después de subir.

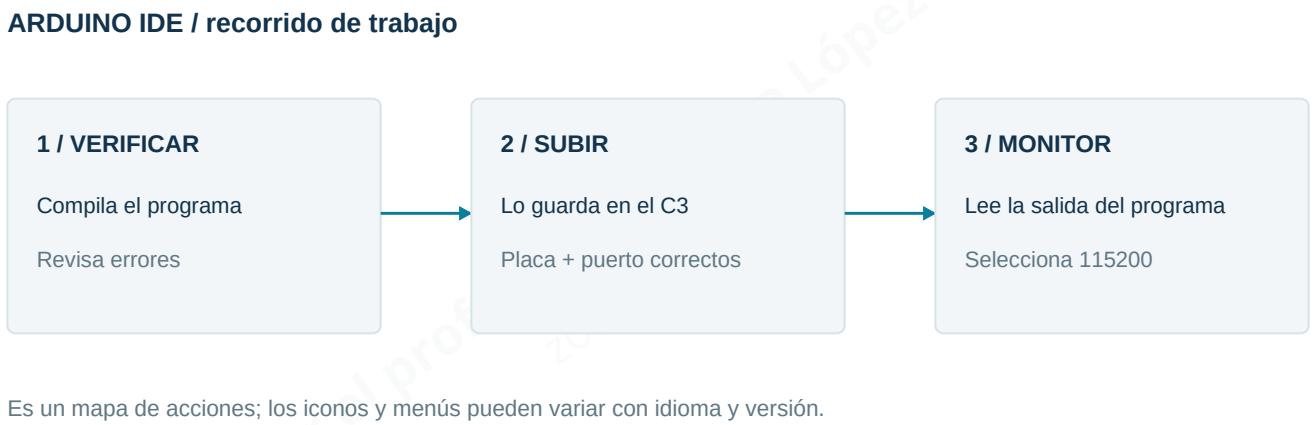


Figura 12. Representación de controles y campos de IDE 2.x; los textos y posiciones pueden variar con idioma/versión. No se presenta como captura real.

4. No instalar controladores al azar. El USB nativo usa CDC/JTAG del C3; una placa con puente CH340/CP210x requiere identificar ese chip y el controlador de su fabricante. En Linux comprobar permisos del puerto con el administrador del laboratorio. Probar primero otro cable de datos y otro puerto USB.

SECCIÓN II / SOFTWARE · CINCO PRÁCTICAS

P1 · Crear, compilar y observar

5. Archivo > Abrir: seleccionar Codigo/Practica_01/Practica_01.ino. Arduino exige que el archivo principal tenga el mismo nombre que la carpeta. Alternativamente crear un sketch nuevo, guardar como Practica_01 y copiar todo el código de las páginas siguientes.

6. Pulsar Verificar (check). Compilar traduce C++ y enlaza bibliotecas; puede hacerse sin placa. Leer el primer error real, no solo “exit status 1”. Pulsar Subir (flecha): ademas transmite el binario. El progreso termina con reinicio o confirmacion de subida. No desconectar durante escritura.

7. Abrir Monitor serie a 115200. Presionar RESET una vez si no se ve el mensaje inicial. Si no se conecta el cargador, mantener BOOT, pulsar RESET, soltar RESET y luego BOOT, seleccionar el puerto de arranque y volver a subir. BOOT no es el botón de control del proyecto.

Cómo leer este programa

setup() corre una vez y configura Serial y pines. **loop()** se repite. **ahora=millis()** mide tiempo transcurrido desde encendido. La resta sin signo ahora-cambio resiste el desbordamiento del contador; no es la hora del reloj. El LED automático cambia cada 500 ms: el ciclo completo dura 1 s.

controles() distingue lectura cruda de estado estable: cuando cambia, inicia una espera de 30 ms sin detener loop(). Si el nuevo nivel persiste, acepta una pulsacion. Se conmuta el modo solo al pasar a LOW, por lo que mantener presionado no lo alterna repetidamente. En manual, enviar la letra **l** cambia la luz; el salto de línea posterior se ignora.

Prueba guiada y resultado

Encender: LED parpadea y aparece modo:1 cada 250 ms. Presionar y soltar: aparece MANUAL y modo:0. Escribir l y enviar: el LED cambia de estado. Volver a pulsar: AUTO y parpadeo. Si el LED no responde, revisar polaridad y GPIO10; si el modo oscila, revisar botón y conexiones antes de aumentar el antirrebote.

Analizar y avanzar

Cronometrar diez ciclos del LED y calcular su período promedio. Cambiar 500 por 250: predecir primero el período completo. Volver al valor original para conservar la referencia de las siguientes prácticas.

> **Verificación.** Compila y sube / Serial legible / Una pulsacion cambia una vez / Modo manual controla LED

SECCIÓN II / SOFTWARE · CINCO PRÁCTICAS

Código completo 01 · 1/1

Practica_01.ino · Líneas 1-37. Copiar los bloques en este orden en un solo archivo; los números impresos son solo una guía.

```
// K9 CALM | Practica 1 | Arduino-ESP32 3.3.4
// LED externo activo en HIGH. No se usa LED integrado.
#include <Arduino.h>
#include <math.h>
constexpr int LED=10, BTN=3;
bool automatico=true, manualLed=false;
uint32_t ahora=0, cambio=0, informe=0;
bool crudo=HIGH, estable=HIGH;
int evento=0, puntos=0; // 0 normal, 1 sonido, 2 posible reaccion, 3 insuficiente
uint32_t alertaHasta=0;
void controles() {
  bool b=digitalRead(BTN);
  if(b!=crudo){ crudo=b; cambio=ahora; }
  if(ahora-cambio>=30 && b!=estable){
    estable=b;
    if(b==LOW){ automatico=!automatico; Serial.println(automatico?"AUTO":"MANUAL"); }
  }
  // En modo manual, enviar l (ele minuscula) conmuta el LED.
  if(Serial.available() && Serial.read()=='l') manualLed=!manualLed;
}

void setup(){
  Serial.begin(115200); // sin esperar al monitor: funciona con pilas
  pinMode(LED,OUTPUT); pinMode(BTN,INPUT_PULLUP);
  Serial.println("K9 CALM listo; boton cambia modo; l conmuta LED manual");
}
void loop(){
  ahora=millis(); controles();
  bool luz=automatico?((ahora/500)%2):manualLed;
  digitalWrite(LED,luz);
  if(ahora-informe>=250){
    informe=ahora;
    Serial.printf("modo:%d",automatico);
    Serial.println();
  }
  yield(); // cede CPU; no espera milisegundos fijos
}
```

SECCIÓN II / SOFTWARE · CINCO PRÁCTICAS

02 / Medir el movimiento

Objetivo. Agregar el MPU6050 y mantener botón, modos, LED y mensajes.

Lo que aprenderemos

Dirección I2C, lecturas con unidades, norma vectorial, filtrado y muestreo periódico.

Materiales y punto de partida

H1 más GY-521; objeto rígido o peluche como soporte. Alimentación USB.

Conservar H1 y agregar H2.

SDA y SCL necesitan pull-ups a 3V3; comprueba los del módulo.

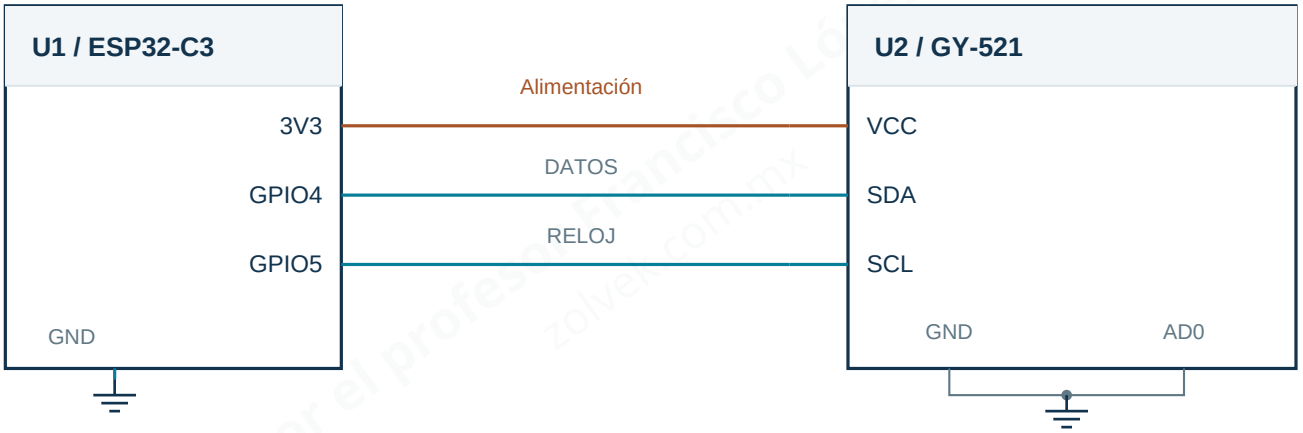


Figura P2. Mismas redes que en hardware; esquema funcional, no huella física.

Red	Conexión
Datos / reloj	4 -> SDA; 5 -> SCL
Energía / dirección	3V3 -> VCC; GND -> GND y AD0

Hecho por el profesor Francisco López Velázquez. zolvek.com.mx

SECCIÓN II / SOFTWARE · CINCO PRÁCTICAS

P2 · Instalar y comprender

En el Gestor de bibliotecas buscar **Adafruit MPU6050 2.2.9** e instalar con todas sus dependencias. Verificar **Adafruit BusIO 1.17.4** y **Adafruit Unified Sensor 1.1.15**. BusIO gestiona intercambios I2C; Unified Sensor define estructuras con unidades. Wire.h ya viene con el core ESP32: no descargar una copia externa.

Abrir Practica_02.ino, verificar placa y puerto, compilar y subir. El programa es autonomo y contiene las funciones de P1; no juntar archivos .ino de distintas prácticas porque se duplicarian setup() y loop(). Al arrancar, el escáner consulta direcciones del bus y muestra las que reconocen el intercambio. Después mpu.begin(0x68,&Wire;) verifica el sensor.

Se configuran rangos de ±8 g y ±500 grados/s para disponer de margen. La biblioteca convierte a m/s² y rad/s. Un rango mayor evita saturación, pero reduce detalle. El filtro interno de 21 Hz ayuda antes de leer a 50 Hz; no equivale a un filtro perfecto contra todas las frecuencias altas.

Datos y funciones nuevas

Módulo / variable	Interpretacion y comprobación
iniciarIMU()	Define 4/5, limita espera I2C y escanea. Falla sin bloquear todo el programa.
movimiento()	Cada 20 ms obtiene a,g,t; si getEvent falla, no actualiza historial.
magnitud	Norma de aceleración: cerca de 9.81 en reposo.
giro	Norma de velocidad angular: cerca de cero quieto, aumenta al rotar.
actividad	Desviación de la norma respecto a gravedad, suavizada. LED AUTO si >0.7.
historial / ultimoIMU	Conservan 100 lecturas para P5 y tiempo de última lectura valida.

Precaución. Sin IMU el programa continua con botón y mensajes; no interpretar act=0 como reposo válido. Revisar “MPU AUSENTE” y volver a encender una vez reparado el cableado.

Abrir Herramientas > Serial Plotter a 115200 para comparar las etiquetas numéricas. No abrir Monitor y Plotter simultáneamente si el puerto queda ocupado. La consola se imprime a 4 Hz para mantener legibilidad, pero el sensor se consulta a 50 Hz: la gráfica serie no conserva todos los picos. Para analizar frecuencia usar registros más rapidos y quitar texto extra, sin cambiar el período de muestreo.

SECCIÓN II / SOFTWARE · CINCO PRÁCTICAS

P2 · Medir, comparar y registrar

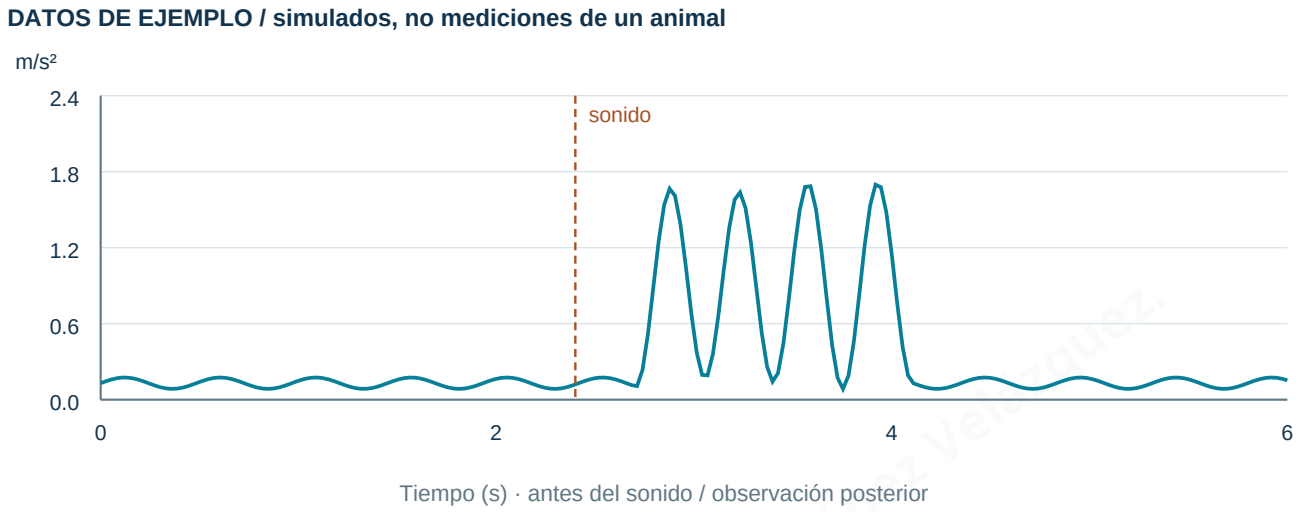


Figura 13. Actividad simulada antes y después de un evento; no es una medición de un perro.

1. Dejar el módulo fijo y quieto 5 s. Registrar a, giro y act. 2. Inclinar lentamente 90 grados sin golpear: los componentes cambian, la norma debe seguir cerca de 9.81. 3. Mover el objeto adelante y atrás unas dos veces por segundo; observar picos. 4. Girarlo suavemente sin trasladarlo; comparar actividad y giro. No lanzar ni golpear sensores.

Ensayo	Esperado	Registrar
Mesa fija	a ~ 9.81; giro pequeño; act baja	Promedio y variación
Inclinación lenta	Norma casi constante	Por qué un eje solo confunde
Sacudida moderada	Actividad sube; LED en AUTO	Valor máximo y tiempo
Giro suave	Giro sube aun con act baja	Limitación de la magnitud

Si algo falla

Sin 0x68: verificar GND, SDA/SCL intercambiados y AD0. 0x69 significa otra selección de dirección; volver AD0 a GND. Datos constantes: comprobar que el módulo realmente se mueve. Valores extremos fijos sugieren saturación, alimentación incorrecta o datos inválidos. Un cable suelto puede parecer un movimiento brusco.

Actividad de análisis

Hacer cinco oscilaciones lentas y cinco rápidas. Estimar frecuencia como ciclos/segundos. Explicar por que un detector de “act alta” no distingue sobresalto de juego ni reconoce todo giro.

Para avanzar: 0x68 reconocido / unidades correctas / 50 Hz configurados / botón y modo manual conservados.

SECCIÓN II / SOFTWARE · CINCO PRÁCTICAS

Código completo 02 · 1/2

Practica_02.ino · Líneas 1-48. Copiar los bloques en este orden en un solo archivo; los números impresos son solo una guía.

```
// K9 CALM | Practica 2 | Arduino-ESP32 3.3.4
// LED externo activo en HIGH. No se usa LED integrado.
#include <Arduino.h>
#include <math.h>
constexpr int LED=10, BTN=3;
bool automatico=true, manualLed=false;
uint32_t ahora=0, cambio=0, informe=0;
bool crudo=HIGH, estable=HIGH;
int evento=0, puntos=0; // 0 normal, 1 sonido, 2 posible reaccion, 3 insuficiente
uint32_t alertaHasta=0;
void controles() {
  bool b=digitalRead(BTN);
  if(b!=crudo){ crudo=b; cambio=ahora; }
  if(ahora-cambio>=30 && b!=estable){
    estable=b;
    if(b==LOW){ automatico=!automatico; Serial.println(automatico?"AUTO":"MANUAL"); }
  }
  // En modo manual, enviar l (ele minuscula) conmuta el LED.
  if(Serial.available() && Serial.read()=='l') manualLed=!manualLed;
}

#include <Wire.h>
#include <Adafruit_MPU6050.h>
Adafruit_MPU6050 mpu;
bool imuOK=false;
float actividad=0, giro=0, magnitud=0;
float historial[100]={0}; // 2 s a 50 Hz; contiene actividad filtrada
int cabeza=0, cuantos=0;
uint32_t ultimoIMU=0, pasoIMU=0;
void iniciarIMU(){
  Wire.begin(4,5); Wire.setTimeOut(10); // SDA, SCL, limite de espera I2C
  Serial.println("Escaner I2C: se espera 0x68");
  for(int d=1;d<127;d++){
    Wire.beginTransmission(d);
    if(Wire.endTransmission()==0) Serial.printf("I2C 0x%02X\n",d);
  }
  imuOK=mpu.begin(0x68,&Wire); // AD0 a GND. Si falla, revisar y reiniciar.
  if(!imuOK){ Serial.println("MPU AUSENTE: revisar H2"); return; }
  mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
  mpu.setGyroRange(MPU6050_RANGE_500_DEG);
  mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);
}
void movimiento(){
  if(!imuOK || ahora-pasoIMU<20) return;
  pasoIMU=ahora;
  sensors_event_t a,g,t;
  if(!mpu.getEvent(&a,&g,&t)) return;
  // Unidades de Adafruit: m/s2 y rad/s (no grados/s).
```

SECCIÓN II / SOFTWARE · CINCO PRÁCTICAS

Código completo 02 · 2/2

Practica_02.ino · Líneas 49-78. Copiar los bloques en este orden en un solo archivo; los números impresos son solo una guía.

```
49    magnitud=sqrtf(a.acceleration.x*a.acceleration.x+
50        a.acceleration.y*a.acceleration.y+a.acceleration.z*a.acceleration.z);
51    giro=sqrtf(g.gyro.x*g.gyro.x+g.gyro.y*g.gyro.y+g.gyro.z*g.gyro.z);
52    float dinamica=fabsf(magnitud-9.80665f);
53    actividad+=0.15f*(dinamica-actividad); // filtro exponencial
54    historial[cabeza]=actividad; cabeza=(cabeza+1)%100;
55    if(cuantos<100) cuantos++;
56    ultimoIMU=ahora;
57  }
58
59  void setup(){
60    Serial.begin(115200); // sin esperar al monitor: funciona con pilas
61    pinMode(LED,OUTPUT); pinMode(BTN,INPUT_PULLUP);
62    iniciarIMU();
63    Serial.println("K9 CALM listo; boton cambia modo; l conmuta LED manual");
64  }
65  void loop(){
66    ahora=millis(); controles();
67    movimiento();
68    bool luz=automatico?((ahora/500)%2):manualLed;
69    if(automatico) luz=(actividad>0.7f);
70    digitalWrite(LED,luz);
71    if(ahora-informe>=250){
72      informe=ahora;
73      Serial.printf("modo:%d",automatico);
74      Serial.printf(" a:%.3f giro:%.3f act:%.3f",magnitud,giro,actividad);
75      Serial.println();
76    }
77    yield(); // cede CPU; no espera milisegundos fijos
78  }
```

Hecho por el profesor Francisco López Velázquez. zolvek.com.mx

SECCIÓN II / SOFTWARE · CINCO PRÁCTICAS

03 / Escuchar y analizar

Objetivo. Añadir audio I2S y conservar el MPU6050 y controles.

Lo que aprenderemos

Muestras con signo, ventanas, RMS, pico, ataque, decaimiento y espectro.

Materiales y punto de partida

H2 más INMP441, 100k de SD a GND y desacoplo local. USB y sonido moderado.

Conservar H2 y agregar H3.

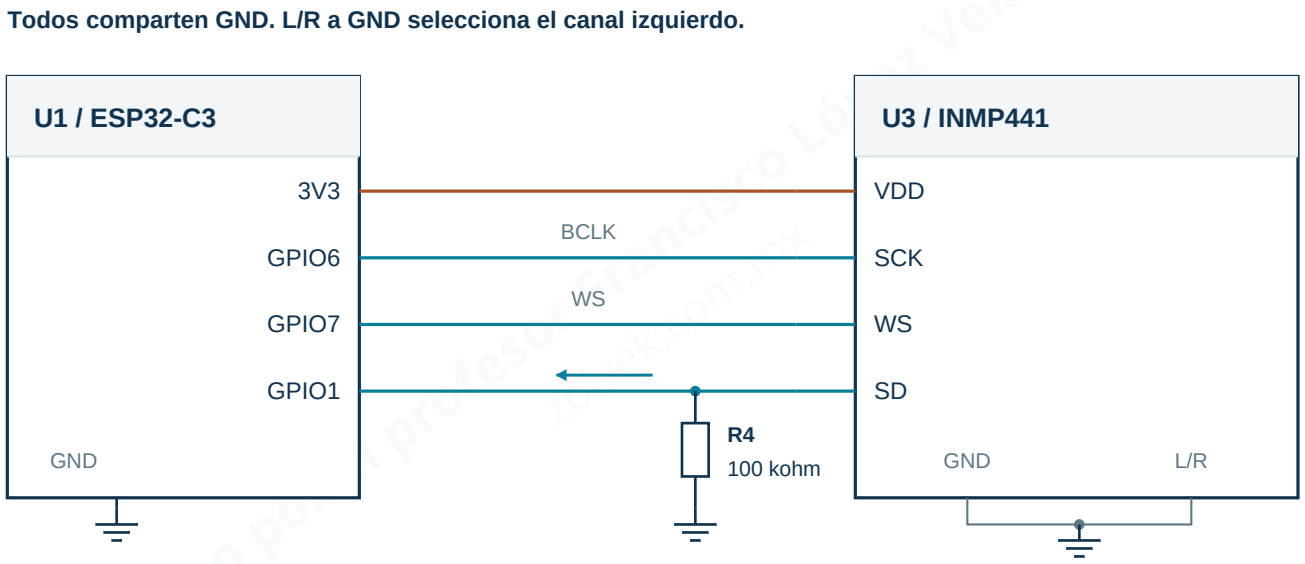


Figura P3. Mismas redes que en hardware; esquema funcional, no huella física.

Red	Conexión
Relojes / datos	6 -> SCK; 7 -> WS; 1 <- SD
Canal / energía	L/R -> GND; VDD -> 3V3; GND común

SECCIÓN II / SOFTWARE · CINCO PRÁCTICAS

P3 · Una ventana de audio

driver/i2s_std.h viene con Arduino-ESP32 3.3.4; no instalar una biblioteca I2S adicional. Abrir P3 y verificar. Esta versión usa el controlador estándar de ESP-IDF, no la API heredada driver/i2s.h. No mezclarlas. El código fija ranuras estereo de 32 bits, aunque solo procesa el canal izquierdo.

DMA guarda muestras mientras la CPU atiende otros módulos. escuchar() pide hasta 64 pares; timeout 0 regresa sin esperar datos nuevos. Va acumulando 256 muestras izquierdas: 256/16000 = **16 ms** por ventana. Si loop se demora demasiado, DMA puede perder datos: no añadir impresiones por muestra ni tareas largas.

RMS paso a paso

Se calcula la media para quitar el componente continuo; después **RMS=sqrt((x1²+…+xN²)/N)**. Para [0.1,-0.1,0.1,-0.1], la media es 0 y el RMS 0.1. El pico es 0.1. Un pico aislado puede tener pico alto y RMS pequeño; la relación pico/RMS ayuda a reconocer impulsos. El silencio digital absoluto no permite dividir: se usa un mínimo 0.000001.

Ataque=max(0,RMSactual-RMSanterior) y **decaimiento=max(0,RMSanterior-RMSactual)** son cambios entre ventanas, no pendientes en unidades por segundo. La duracion avanza mientras la señal permanece sobre el umbral; al caer, conserva el último valor. Es aproximada a 16 ms y puede separar colas intermitentes.

Detector funcional antes del espectro

El ruido se estima lentamente en ventanas no fuertes. Un sonido fuerte exige RMS>max(0.004,6×ruido). Un impulso ademas exige pico/RMS>2, ataque>2×ruido y más de 2 s desde el último. El primer segundo calienta el estimador. Primero observar RMS/pico y probar el LED; la FFT solo añade una descripcion, no es necesaria para encenderlo.

> **Precaución.** Estos umbrales se ajustan con datos de banco. No reproducir pirotecnia real ni hacer sonidos intensos cerca de personas o animales. Una grabacion moderada puede no superar el umbral: ajustar con criterio, no aumentar volumen indefinidamente.

SECCIÓN II / SOFTWARE · CINCO PRÁCTICAS

P3 · FFT sin magia

DATOS DE EJEMPLO / simulados, no mediciones de un animal

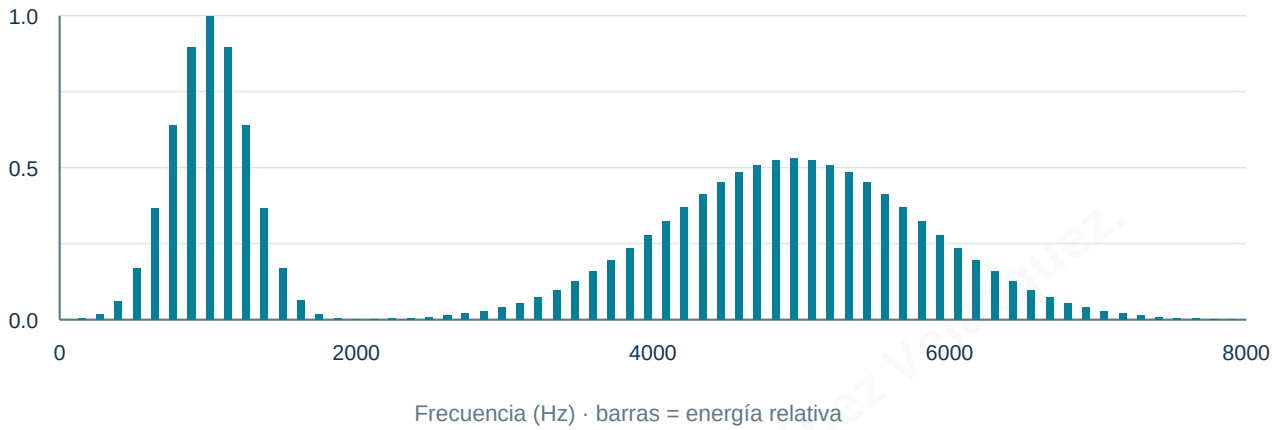


Figura 14. Espectro sintético: las barras representan energía relativa, no una clasificación del origen.

La FFT reorganiza las muestras para estimar cuanto contenido hay en diferentes frecuencias. Una analogia es separar los ingredientes de una mezcla. Nuestro algoritmo radix-2 necesita N potencia de dos. Con N=256 y Fs=16000, la separacion entre bins es **62.5 Hz**; el bin 32 corresponde a 2000 Hz. Solo usamos frecuencias positivas por debajo de 8000 Hz, límite de Nyquist.

La ventana Hann reduce fugas al cortar una señal continua en bloques: multiplica cada muestra por 0.5-0.5 cos(2πi/(N-1)). Se ponen a cero las partes imaginarias, se reordenan indices y se combinan pares en etapas. La energía relativa de un bin es re^2+im^2. Se suman bins 2-127 y se divide la energía desde 2 kHz entre ese total: **agudos** va de 0 a 1. No se interpreta como porcentaje de “miedo”.

Cómo incorporarlo de forma guiada

1. Compilar P3 completo y confirmar que RMS y pico reaccionan. 2. Localizar espectro(): se llama después de guardar RMS y ataque porque modifica re[]. 3. Observar ag en Serial con voz grave y un sonido agudo suave. 4. Para aislar una falla, comentar solo espectro(); RMS e impulsos deben seguir funcionando y ag queda en cero. Restaurar la llamada antes de pasar a P4.

La FFT describe solo una ventana de 16 ms y no el evento completo. Un sonido grave puede tener ataque con energía alta; el montaje y la distancia cambian la señal. El detector conserva el movimiento y permite comparar coincidencias temporales, pero no reconoce petardos de manera validada.

SECCIÓN II / SOFTWARE · CINCO PRÁCTICAS

P3 · Ensayo acústico y lectura del código

Función	Que hace / comprobar
iniciarAudio()	Crea canal RX, configura reloj y GPIO 6/7/1 y activa DMA. audioOK confirma inicializacion, no sonido correcto.
escuchar()	Recibe pares, descarta ranura derecha, normaliza y llena re[].
analizarAudio()	Quita media, calcula RMS/pico/ataque/decaimiento y compara con umbral.
espectro()	Hann + FFT in situ + proporción de energía >=2kHz.
impulso / ultimoAudio	Pulso de una vuelta de loop y marca temporal de última ventana procesada.

Tras subir, esperar 3 s en silencio relativo. Hablar suave y comprobar que rms y pico varian sin saturar. Reproducir una palmada grabada a volumen moderado, esperar 3 s y repetir. Comparar con un portazo, pirotecnia y trueno grabados, sin producir los sonidos originales. Registrar distancia y volumen para repetir condiciones. Mover luego el objeto suavemente y observar act en el mismo registro.

Muestra	RMS / pico	ag / dur	¿Impulso? / movimiento
Voz / ambiente	_____	_____	_____
Aplauso grabado	_____	_____	_____
Portazo grabado	_____	_____	_____
Pirotecnia / trueno grabados	_____	_____	_____

Si rms=0 siempre: revisar L/R a GND, SD en GPIO1, 3.3 V y relojes. Si pico ~ 1 sostenido: revisar formato, interferencia y volumen. Si responde solo al mover cables, corregir contactos. No tratar valores de inicializacion como mediciones validas.

Actividad y verificación

Comparar una palmada breve con una vocal sostenida de intensidad moderada: ¿cual tiene mayor pico/RMS?, ¿el umbral fijo genera falsas detecciones? Cambiar una sola constante a la vez y registrar el efecto.

Verificación. RMS dinamico / No saturación permanente / MPU sigue midiendo / FFT restaurada / Modos y LED conservados

SECCIÓN II / SOFTWARE · CINCO PRÁCTICAS

Código completo 03 · 1/4

Practica_03.ino · Líneas 1-48. Copiar los bloques en este orden en un solo archivo; los números impresos son solo una guía.

```
// K9 CALM | Practica 3 | Arduino-ESP32 3.3.4
// LED externo activo en HIGH. No se usa LED integrado.
#include <Arduino.h>
#include <math.h>
constexpr int LED=10, BTN=3;
bool automatico=true, manualLed=false;
uint32_t ahora=0, cambio=0, informe=0;
bool crudo=HIGH, estable=HIGH;
int evento=0, puntos=0; // 0 normal, 1 sonido, 2 posible reaccion, 3 insuficiente
uint32_t alertaHasta=0;
void controles() {
  bool b=digitalRead(BTN);
  if(b!=crudo){ crudo=b; cambio=ahora; }
  if(ahora-cambio>=30 && b!=estable){
    estable=b;
    if(b==LOW){ automatico=!automatico; Serial.println(automatico?"AUTO":"MANUAL"); }
  }
  // En modo manual, enviar l (ele minuscula) conmuta el LED.
  if(Serial.available() && Serial.read()=='l') manualLed=!manualLed;
}

#include <Wire.h>
#include <Adafruit_MPU6050.h>
Adafruit_MPU6050 mpu;
bool imuOK=false;
float actividad=0, giro=0, magnitud=0;
float historial[100]={0}; // 2 s a 50 Hz; contiene actividad filtrada
int cabeza=0, cuantos=0;
uint32_t ultimoIMU=0, pasoIMU=0;
void iniciarIMU(){
  Wire.begin(4,5); Wire.setTimeOut(10); // SDA, SCL, limite de espera I2C
  Serial.println("Escaner I2C: se espera 0x68");
  for(int d=1;d<127;d++){
    Wire.beginTransmission(d);
    if(Wire.endTransmission()==0) Serial.printf("I2C 0x%02X\n",d);
  }
  imuOK=mpu.begin(0x68,&Wire); // AD0 a GND. Si falla, revisar y reiniciar.
  if(!imuOK){ Serial.println("MPU AUSENTE: revisar H2"); return; }
  mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
  mpu.setGyroRange(MPU6050_RANGE_500_DEG);
  mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);
}
void movimiento(){
  if(!imuOK || ahora-pasoIMU<20) return;
  pasoIMU=ahora;
  sensors_event_t a,g,t;
  if(!mpu.getEvent(&a,&g,&t)) return;
  // Unidades de Adafruit: m/s2 y rad/s (no grados/s).
```

SECCIÓN II / SOFTWARE · CINCO PRÁCTICAS

Código completo 03 · 2/4

Practica_03.ino · Líneas 49-96. Copiar los bloques en este orden en un solo archivo; los números impresos son solo una guía.

```
  magnitud=sqrtf(a.acceleration.x*a.acceleration.x+
      a.acceleration.y*a.acceleration.y+a.acceleration.z*a.acceleration.z);
  giro=sqrtf(g.gyro.x*g.gyro.x+g.gyro.y*g.gyro.y+g.gyro.z*g.gyro.z);
  float dinamica=fabsf(magnitud-9.80665f);
  actividad+=0.15f*(dinamica-actividad); // filtro exponencial
  historial[cabeza]=actividad; cabeza=(cabeza+1)%100;
  if(cuantos<100) cuantos++;
  ultimoIMU=ahora;
}

#include <driver/i2s_std.h>
i2s_chan_handle_t rx=nullptr;
bool audioOK=false, impulso=false;
constexpr int N=256, FS=16000;
float re[N], im[N], rms=0, pico=0, ruido=0.001f, agudos=0;
float ataque=0, decaimiento=0, rmsAnterior=0;
uint32_t ultimoAudio=0, inicioSonido=0, duracion=0, ultimoPulso=0;
bool sonando=false;
int llenos=0;
void iniciarAudio(){
  i2s_chan_config_t c=I2S_CHANNEL_DEFAULT_CONFIG(I2S_NUM_0,I2S_ROLE_MASTER);
  c.dma_desc_num=8; c.dma_frame_num=128;
  if(i2s_new_channel(&c,nullptr,&rx)!=ESP_OK) return;
  i2s_std_config_t s={};
  s.clk_cfg=I2S_STD_CLK_DEFAULT_CONFIG(FS);
  // Dos ranuras de 32 bits = 64 BCLK por trama. Solo usamos izquierda.
  s.slot_cfg=I2S_STD_PHILIPS_SLOT_DEFAULT_CONFIG(
      I2S_DATA_BIT_WIDTH_32BIT,I2S_SLOT_MODE_STEREO);
  s.gpio_cfg.mclk=I2S_GPIO_UNUSED; s.gpio_cfg.bclk=GPIO_NUM_6;
  s.gpio_cfg.ws=GPIO_NUM_7; s.gpio_cfg.dout=I2S_GPIO_UNUSED;
  s.gpio_cfg.din=GPIO_NUM_1;
  audioOK=(i2s_channel_init_std_mode(rx,&s)==ESP_OK &&
           i2s_channel_enable(rx)==ESP_OK);
  if(!audioOK) Serial.println("ERROR I2S: revisar version y pines");
}
// FFT radix-2 in situ. N debe ser potencia de 2.
void espectro(){
  for(int i=0;i<N;i++){
    re[i]*=0.5f-0.5f*cosf(2*PI*i/(N-1)); im[i]=0; // ventana Hann
  }
  for(int i=1,j=0;i<N;i++){
    int b=N>>1; for(;j&b;b>>=1) j^=b; j^=b;
    if(i<j){ float v=re[i]; re[i]=re[j]; re[j]=v; }
  }
  for(int len=2;len<=N;len<<=1){
    float ang=-2*PI/len;
    for(int i=0;i<N;i+=len){
      for(int j=0;j<len/2;j++){
```

SECCIÓN II / SOFTWARE · CINCO PRÁCTICAS

Código completo 03 · 3/4

Practica_03.ino · Líneas 97-144. Copiar los bloques en este orden en un solo archivo; los números impresos son solo una guía.

```
 97          int u=i+j,v=u+len/2;
 98          float c=cosf(ang*j),s=sinf(ang*j);
 99          float tr=c*re[v]-s*im[v],ti=s*re[v]+c*im[v];
100          re[v]=re[u]-tr; im[v]=im[u]-ti; re[u]+=tr; im[u]+=ti;
101        }
102      }
103    }
104    float total=0,alta=0;
105    for(int k=2;k<N/2;k++){
106      float e=re[k]*re[k]+im[k]*im[k]; total+=e;
107      if(k*FS/N>=2000) alta+=e;
108    }
109    agudos=total>1e-12f?alta/total:0; // proporcion, no identificador del sonido
110  }
111  void analizarAudio(){
112    float media=0, suma=0; pico=0;
113    for(int i=0;i<N;i++) media+=re[i]/N;
114    for(int i=0;i<N;i++){
115      re[i]-=media; suma+=re[i]*re[i]; pico=fmaxf(pico,fabsf(re[i]));
116    }
117    rms=sqrtf(suma/N);
118    ataque=fmaxf(0,rms-rmsAnterior); decaimiento=fmaxf(0,rmsAnterior-rms);
119    float umbral=fmaxf(0.004f,ruido*6);
120    bool fuerte=(rms>umbral);
121    // Calentamiento 1 s; no se aprende el ruido durante sonidos fuertes.
122    if(ahora<1000 || !fuerte) ruido+=0.01f*(rms-ruido);
123    impulso=(ahora>=1000 && fuerte && !sonando &&
124        pico/fmaxf(rms,1e-6f)>2.0f && ataque>ruido*2 &&
125        ahora-ultimoPulso>2000);
126    if(impulso) ultimoPulso=ahora;
127    if(fuerte && !sonando){ inicioSonido=ahora; sonando=true; }
128    if(sonando) duracion=ahora-inicioSonido;
129    if(!fuerte) sonando=false;
130    rmsAnterior=rms;
131    espectro(); // destruye re[]; se vuelve a llenar en la siguiente ventana
132    ultimoAudio=ahora;
133  }
134  void escuchar(){
135    impulso=false;
136    if(!audioOK) return;
137    int32_t par[64*2]; size_t leidos=0;
138    // Timeout 0: si DMA no tiene datos, volvemos sin esperar.
139    i2s_channel_read(rx,par,sizeof(par),&leidos,0);
140    for(size_t j=0;j+1<leidos/4;j+=2){
141      // 24 bits validos alineados a la izquierda; canal L/R conectado a GND.
142      re[llenos++]=(par[j]>>8)/8388608.0f;
143      if(llenos==N){ llenos=0; analizarAudio(); }
144    }
```

SECCIÓN II / SOFTWARE · CINCO PRÁCTICAS

Código completo 03 · 4/4

Practica_03.ino · Líneas 145-172. Copiar los bloques en este orden en un solo archivo; los números impresos son solo una guía.

```
145  }
146
147  void setup(){
148    Serial.begin(115200); // sin esperar al monitor: funciona con pilas
149    pinMode(LED,OUTPUT); pinMode(BTN,INPUT_PULLUP);
150    iniciarIMU();
151    iniciarAudio();
152    Serial.println("K9 CALM listo; boton cambia modo; l conmuta LED manual");
153  }
154  void loop(){
155    ahora=millis(); controles();
156    movimiento();
157    escuchar();
158    if(impulso){ evento=1; alertaHasta=ahora+1000; }
159    bool luz=automatico?((ahora/500)%2):manualLed;
160    if(automatico) luz=(actividad>0.7f);
161    if(automatico && int32_t(alertaHasta-ahora)>0) luz=true;
162    digitalWrite(LED,luz);
163    if(ahora-informe>=250){
164      informe=ahora;
165      Serial.printf("modo:%d",automatico);
166      Serial.printf(" a:%.3f giro:%.3f act:%.3f",magnitud,giro,actividad);
167      Serial.printf(" rms:%.5f pico:%.5f ag:%.2f dur:%lu",rms,pico,agudos,
168          (unsigned long)duracion);
169      Serial.println();
170    }
171    yield(); // cede CPU; no espera milisegundos fijos
172  }
```

Hecho por el profesor Francisco López Velázquez. zolvek.com.mx

SECCIÓN II / SOFTWARE · CINCO PRÁCTICAS

04 / Batería y Bluetooth

Objetivo. Publicar estado en un teléfono y ensayar la medida de entrada protegida.

Lo que aprenderemos

ADC calibrado, histéresis, servicios GATT, lectura y notificaciones BLE.

Materiales y punto de partida

H3; Android con BLE y nRF Connect. Para medir: H4, multímetro y fuente limitada. PILAS=false al comenzar.

Conservar H3 y agregar H4.

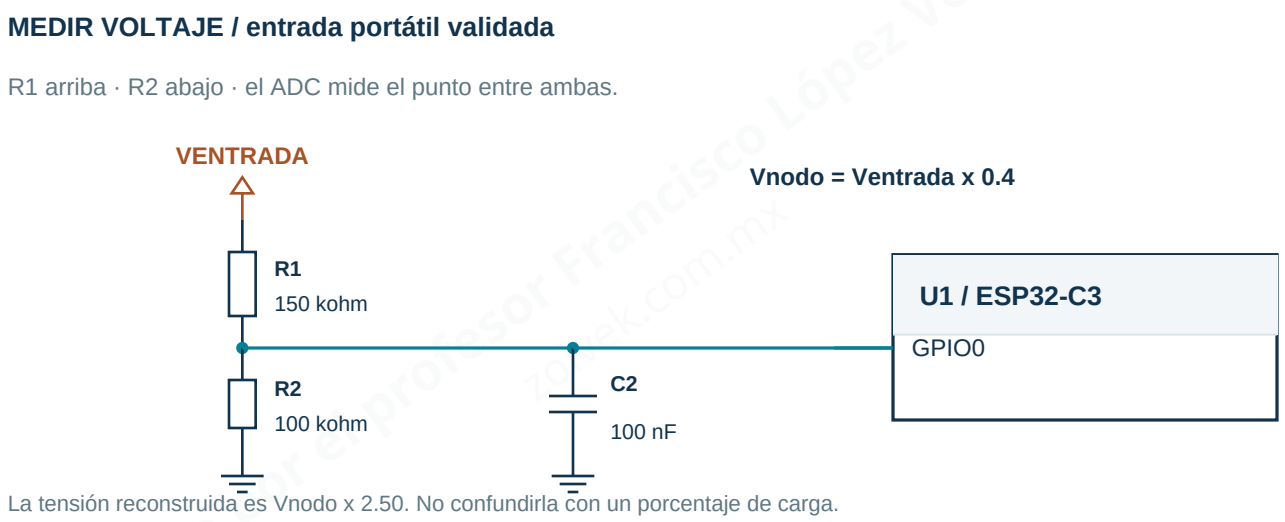


Figura P4. Mismas redes que en hardware; esquema funcional, no huella física.

Red	Conexión
Medida	Nodo S1 -> 150k -> GPIO0 -> 100k -> GND
Filtro	100nF: GPIO0 a GND; PILAS=false en USB

SECCIÓN II / SOFTWARE · CINCO PRÁCTICAS

P4 · BLE y sus límites

Instalar desde Gestor de bibliotecas **NimBLE-Arduino 2.5.2** de h2zero. Se usa porque ofrece BLE con menor carga de recursos que alternativas basadas en el stack clásico. No instalar otra NimBLE ni reemplazarla con BLESerial. Las firmas de callbacks varian entre versiones: el código está fijado a 2.5.2. Abrir P4, compilar y subir conservando bibliotecas de P2.

BLE transmite pequeños datos con bajo consumo relativo. El ESP32 anuncia su nombre; Android actua como central. Un **servicio** agrupa datos y una **característica** contiene un valor y permisos. Una UUID identifica cada uno. “Leer” solicita el valor actual; “notificar” permite recibir cambios después de suscribirse. Una notificación GATT no crea por si misma una alerta del sistema operativo ni un sonido con la pantalla bloqueada.

UUID (mismo sufijo)	Contenido / permiso
8d6c0001-3d68-4b85-9b71-4d3943414c4d	Servicio K9 CALM
8d6c0002-3d68-4b85-9b71-4d3943414c4d	Estado ASCII: READ y NOTIFY, cada 500 ms
8d6c0003-3d68-4b85-9b71-4d3943414c4d	Voltaje ASCII: READ; “USB; NA” si PILAS=false

Ejemplo **A E2 B0 Q80**: A automático (M manual); E0 normal, E1 sospechoso, E2 posible reacción, E3 insuficiente; B0 normal/USB, B1 baja, B2 crítica, B3 ADC inválido; Q puntuación experimental 0-100. El estado ocupa menos de 20 bytes y funciona con MTU ATT mínimo de 23. La característica de voltaje distingue USB de batería real.

No se publica Battery Service 0x180F/2A19 porque exige un porcentaje que no hemos validado para estas AAA. Se comunica voltaje y estado mediante servicio propio. No se transmite audio ni se guarda información de mascotas. El ejemplo no exige autenticacion: usarlo en laboratorio, sin datos personales.

SECCIÓN II / SOFTWARE · CINCO PRÁCTICAS

P4 · Conectarse desde Android

1. Instalar **nRF Connect for Mobile** de Nordic Semiconductor desde la tienda oficial; verificar el editor. Activar Bluetooth y conceder “dispositivos cercanos”; Android antiguo puede pedir ubicacion para escanear. No es necesario emparejar desde Ajustes para este ejemplo.

2. Subir P4 y abrir nRF Connect > Scanner > Scan. Buscar K9-CALM; acercar el teléfono a un metro. Si el nombre no aparece, buscar la UUID de servicio; revisar que el escaneo activo permita respuesta de scan. Solo un teléfono debe conectarse para simplificar el ensayo.

3. Pulsar Connect y expandir el servicio personalizado. En la característica terminada en 0002, pulsar lectura y elegir visualizacion UTF-8/Text. Activar notificaciones con el control de suscripción; su icono puede variar. Debe recibirse un valor aproximadamente cada 500 ms.

4. Pulsar el botón físico: A cambia a M. En la característica 0003 leer “USB; NA”. Reproducir el sonido de P3: P4 comunica E1, aun sin comparar movimiento posterior. Desconectar desde la app y volver a buscar: el callback reinicia la publicidad.

Lo que se incorpora al código

iniciarBLE() configura servidor, servicio, características y anuncio. **publicar()** actualiza las dos características sin imprimir muestras individuales; notify() solicita una entrega, no garantiza que la app este suscrita. **Conexión::onDisconnect()** usa la firma de NimBLE 2.5.2 y vuelve a anunciar.

medirPilas() corre cada segundo solo si PILAS=true. Promedia 16 valores en mV, reconstruye voltaje y clasifica con histéresis. B3 indica lectura fuera del rango didactico 100-2400 mV, no cualquier falla eléctrica. Un divisor mal montado puede aparentar voltaje válido: el multímetro sigue siendo necesario.

> **Información.** En P4, E1 queda como memoria del último impulso hasta reiniciar; el pulso del LED dura 1 s. En P5 el estado vuelve a normal tras la espera. Esta diferencia prepara la máquina de estados.

SECCIÓN II / SOFTWARE · CINCO PRÁCTICAS

P4 · Ensayar ADC y alarmas

Primero mantener USB y pack desconectado. Para probar el divisor sin energizar dos veces la placa: separar fisicamente el cable Ventrada -> pin 5V. Conectar la fuente limitada solo a R1 y GND común, medir el nodo y confirmar <=2.4 V. No usar pilas durante este ensayo. Con la placa ya alimentada, variar la fuente de prueba entre 3.6 y 4.8 V, sin superar el rango permitido del nodo.

Cambiar PILAS=true temporalmente para ensayar ADC; compilar y subir. Esperar un segundo por punto y comparar lectura con multímetro en el punto superior R1-GND. Calcular CAL y repetir. Probar entrada de 4.2 V, 3.95 V, 4.05 V y 4.2 V: entra en baja en 3.95 y solo sale por arriba de 4.15. Probar 3.75 V para crítica; el LED parpadea rápido. Estos valores ensayan lógica, no aprueban la alimentación real.

V fuente / multímetro	V ADC esperada	V programa	B / LED
4.20 V	1.680 V	_____	0
3.95 V	1.580 V	_____	1 / lento
4.05 V	1.620 V	_____	1 / permanece
4.20 V	1.680 V	_____	0 / sale
3.75 V	1.500 V	_____	2 / rápido

Al terminar, restaurar PILAS=false para USB normal. Para pasar a AAA, ejecutar toda la validación H4, fijar umbrales reales, conectar el cable de Ventrada a 5V solo sin USB y cargar firmware PILAS=true previamente. Si no hay regulador identificado o estabilidad demostrada, permanecer en modalidad USB.

Análisis y criterios de avance

¿Por que un voltaje bajo no equivale a un porcentaje exacto? ¿Por que la alarma no evita un reinicio por una caída rápida? Registrar el error máximo en tres puntos y distinguir caída de D1 de error ADC. Si BLE no cambia, releer valor y verificar suscripción antes de modificar el código.

Verificación. BLE lee A/M / Suscripción recibe datos / ADC comparado con multímetro / Histéresis comprobada / PILAS=false restaurado en USB

SECCIÓN II / SOFTWARE · CINCO PRÁCTICAS

Código completo 04 · 1/5

Practica_04.ino · Líneas 1-48. Copiar los bloques en este orden en un solo archivo; los números impresos son solo una guía.

```
// K9 CALM | Practica 4 | Arduino-ESP32 3.3.4
// LED externo activo en HIGH. No se usa LED integrado.
#include <Arduino.h>
#include <math.h>
constexpr int LED=10, BTN=3;
bool automatico=true, manualLed=false;
uint32_t ahora=0, cambio=0, informe=0;
bool crudo=HIGH, estable=HIGH;
int evento=0, puntos=0; // 0 normal, 1 sonido, 2 posible reaccion, 3 insuficiente
uint32_t alertaHasta=0;
void controles() {
  bool b=digitalRead(BTN);
  if(b!=crudo){ crudo=b; cambio=ahora; }
  if(ahora-cambio>=30 && b!=estable){
    estable=b;
    if(b==LOW){ automatico=!automatico; Serial.println(automatico?"AUTO":"MANUAL"); }
  }
  // En modo manual, enviar l (ele minuscula) conmuta el LED.
  if(Serial.available() && Serial.read()=='l') manualLed=!manualLed;
}

#include <Wire.h>
#include <Adafruit_MPU6050.h>
Adafruit_MPU6050 mpu;
bool imuOK=false;
float actividad=0, giro=0, magnitud=0;
float historial[100]={0}; // 2 s a 50 Hz; contiene actividad filtrada
int cabeza=0, cuantos=0;
uint32_t ultimoIMU=0, pasoIMU=0;
void iniciarIMU(){
  Wire.begin(4,5); Wire.setTimeOut(10); // SDA, SCL, limite de espera I2C
  Serial.println("Escaner I2C: se espera 0x68");
  for(int d=1;d<127;d++){
    Wire.beginTransmission(d);
    if(Wire.endTransmission()==0) Serial.printf("I2C 0x%02X\n",d);
  }
  imuOK=mpu.begin(0x68,&Wire); // AD0 a GND. Si falla, revisar y reiniciar.
  if(!imuOK){ Serial.println("MPU AUSENTE: revisar H2"); return; }
  mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
  mpu.setGyroRange(MPU6050_RANGE_500_DEG);
  mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);
}
void movimiento(){
  if(!imuOK || ahora-pasoIMU<20) return;
  pasoIMU=ahora;
  sensors_event_t a,g,t;
  if(!mpu.getEvent(&a,&g,&t)) return;
  // Unidades de Adafruit: m/s2 y rad/s (no grados/s).
```

SECCIÓN II / SOFTWARE · CINCO PRÁCTICAS

Código completo 04 · 2/5

Practica_04.ino · Líneas 49-96. Copiar los bloques en este orden en un solo archivo; los números impresos son solo una guía.

```
  magnitud=sqrtf(a.acceleration.x*a.acceleration.x+
      a.acceleration.y*a.acceleration.y+a.acceleration.z*a.acceleration.z);
  giro=sqrtf(g.gyro.x*g.gyro.x+g.gyro.y*g.gyro.y+g.gyro.z*g.gyro.z);
  float dinamica=fabsf(magnitud-9.80665f);
  actividad+=0.15f*(dinamica-actividad); // filtro exponencial
  historial[cabeza]=actividad; cabeza=(cabeza+1)%100;
  if(cuantos<100) cuantos++;
  ultimoIMU=ahora;
}

#include <driver/i2s_std.h>
i2s_chan_handle_t rx=nullptr;
bool audioOK=false, impulso=false;
constexpr int N=256, FS=16000;
float re[N], im[N], rms=0, pico=0, ruido=0.001f, agudos=0;
float ataque=0, decaimiento=0, rmsAnterior=0;
uint32_t ultimoAudio=0, inicioSonido=0, duracion=0, ultimoPulso=0;
bool sonando=false;
int llenos=0;
void iniciarAudio(){
  i2s_chan_config_t c=I2S_CHANNEL_DEFAULT_CONFIG(I2S_NUM_0,I2S_ROLE_MASTER);
  c.dma_desc_num=8; c.dma_frame_num=128;
  if(i2s_new_channel(&c,nullptr,&rx)!=ESP_OK) return;
  i2s_std_config_t s={};
  s.clk_cfg=I2S_STD_CLK_DEFAULT_CONFIG(FS);
  // Dos ranuras de 32 bits = 64 BCLK por trama. Solo usamos izquierda.
  s.slot_cfg=I2S_STD_PHILIPS_SLOT_DEFAULT_CONFIG(
      I2S_DATA_BIT_WIDTH_32BIT,I2S_SLOT_MODE_STEREO);
  s.gpio_cfg.mclk=I2S_GPIO_UNUSED; s.gpio_cfg.bclk=GPIO_NUM_6;
  s.gpio_cfg.ws=GPIO_NUM_7; s.gpio_cfg.dout=I2S_GPIO_UNUSED;
  s.gpio_cfg.din=GPIO_NUM_1;
  audioOK=(i2s_channel_init_std_mode(rx,&s)==ESP_OK &&
           i2s_channel_enable(rx)==ESP_OK);
  if(!audioOK) Serial.println("ERROR I2S: revisar version y pines");
}
// FFT radix-2 in situ. N debe ser potencia de 2.
void espectro(){
  for(int i=0;i<N;i++){
    re[i]*=0.5f-0.5f*cosf(2*PI*i/(N-1)); im[i]=0; // ventana Hann
  }
  for(int i=1,j=0;i<N;i++){
    int b=N>>1; for(;j&b;b>>=1) j^=b; j^=b;
    if(i<j){ float v=re[i]; re[i]=re[j]; re[j]=v; }
  }
  for(int len=2;len<=N;len<<=1){
    float ang=-2*PI/len;
    for(int i=0;i<N;i+=len){
      for(int j=0;j<len/2;j++){
```

SECCIÓN II / SOFTWARE · CINCO PRÁCTICAS

Código completo 04 · 3/5

Practica_04.ino · Líneas 97-144. Copiar los bloques en este orden en un solo archivo; los números impresos son solo una guía.

```
 97          int u=i+j,v=u+len/2;
 98          float c=cosf(ang*j),s=sinf(ang*j);
 99          float tr=c*re[v]-s*im[v],ti=s*re[v]+c*im[v];
100          re[v]=re[u]-tr; im[v]=im[u]-ti; re[u]+=tr; im[u]+=ti;
101        }
102      }
103    }
104    float total=0,alta=0;
105    for(int k=2;k<N/2;k++){
106      float e=re[k]*re[k]+im[k]*im[k]; total+=e;
107      if(k*FS/N>=2000) alta+=e;
108    }
109    agudos=total>1e-12f?alta/total:0; // proporcion, no identificador del sonido
110  }
111  void analizarAudio(){
112    float media=0, suma=0; pico=0;
113    for(int i=0;i<N;i++) media+=re[i]/N;
114    for(int i=0;i<N;i++){
115      re[i]-=media; suma+=re[i]*re[i]; pico=fmaxf(pico,fabsf(re[i]));
116    }
117    rms=sqrtf(suma/N);
118    ataque=fmaxf(0,rms-rmsAnterior); decaimiento=fmaxf(0,rmsAnterior-rms);
119    float umbral=fmaxf(0.004f,ruido*6);
120    bool fuerte=(rms>umbral);
121    // Calentamiento 1 s; no se aprende el ruido durante sonidos fuertes.
122    if(ahora<1000 || !fuerte) ruido+=0.01f*(rms-ruido);
123    impulso=(ahora>=1000 && fuerte && !sonando &&
124        pico/fmaxf(rms,1e-6f)>2.0f && ataque>ruido*2 &&
125        ahora-ultimoPulso>2000);
126    if(impulso) ultimoPulso=ahora;
127    if(fuerte && !sonando){ inicioSonido=ahora; sonando=true; }
128    if(sonando) duracion=ahora-inicioSonido;
129    if(!fuerte) sonando=false;
130    rmsAnterior=rms;
131    espectro(); // destruye re[]; se vuelve a llenar en la siguiente ventana
132    ultimoAudio=ahora;
133  }
134  void escuchar(){
135    impulso=false;
136    if(!audioOK) return;
137    int32_t par[64*2]; size_t leidos=0;
138    // Timeout 0: si DMA no tiene datos, volvemos sin esperar.
139    i2s_channel_read(rx,par,sizeof(par),&leidos,0);
140    for(size_t j=0;j+1<leidos/4;j+=2){
141      // 24 bits validos alineados a la izquierda; canal L/R conectado a GND.
142      re[llenos++]=(par[j]>>8)/8388608.0f;
143      if(llenos==N){ llenos=0; analizarAudio(); }
144    }
```

SECCIÓN II / SOFTWARE · CINCO PRÁCTICAS

Código completo 04 · 4/5

Practica_04.ino · Líneas 145-192. Copiar los bloques en este orden en un solo archivo; los números impresos son solo una guía.

```
145  }
146
147  #include <NimBLEDevice.h>
148  // Cambiar a true SOLO tras validar H4. En USB, desconectar el portapilas.
149  constexpr bool PILAS=false;
150  constexpr float CAL=1.000f, FACTOR=2.50f; // 150k arriba + 100k abajo
151  // Umbrales didacticos: sustituir por los determinados con carga y regulador real.
152  constexpr float V_BAJA=4.00f, V_CRITICA=3.80f, HISTERESIS=0.15f;
153  float voltaje=0; int bateria=0; // 0 USB/normal, 1 baja, 2 critica, 3 ADC invalido
154  uint32_t pasoADC=0, pasoBLE=0;
155  NimBLECharacteristic *estadoBLE=nullptr, *voltajeBLE=nullptr;
156  const char* SERVICIO="8d6c0001-3d68-4b85-9b71-4d3943414c4d";
157  class Conexion:public NimBLEServerCallbacks{
158    void onDisconnect(NimBLEServer*,NimBLEConnInfo&,int) override{
159      NimBLEDevice::startAdvertising();
160    }
161  } conexion;
162  void iniciarBLE(){
163    NimBLEDevice::init("K9-CALM");
164    auto s=NimBLEDevice::createServer(); s->setCallbacks(&conexion,false);
165    auto servicio=s->createService(SERVICIO);
166    estadoBLE=servicio->createCharacteristic(
167        "8d6c0002-3d68-4b85-9b71-4d3943414c4d",
168        NIMBLE_PROPERTY::READ|NIMBLE_PROPERTY::NOTIFY);
169    voltajeBLE=servicio->createCharacteristic(
170        "8d6c0003-3d68-4b85-9b71-4d3943414c4d",NIMBLE_PROPERTY::READ);
171    servicio->start();
172    auto a=NimBLEDevice::getAdvertising();
173    a->setName("K9-CALM"); a->addServiceUUID(SERVICIO);
174    a->enableScanResponse(true); a->start();
175    analogReadResolution(12); analogSetPinAttenuation(0,ADC_11db);
176  }
177  void medirPilas(){
178    if(!PILAS || ahora-pasoADC<1000) return;
179    pasoADC=ahora;
180    uint32_t suma=0;
181    for(int j=0;j<16;j++) suma+=analogReadMilliVolts(0);
182    float mv=suma/16.0f; voltaje=mv*FACTOR*CAL/1000;
183    if(mv<100 || mv>2400){ bateria=3; return; }
184    if(bateria==3) bateria=0;
185    if(voltaje<V_CRITICA) bateria=2;
186    else if(bateria==2 && voltaje<V_CRITICA+HISTERESIS) return;
187    else if(voltaje<V_BAJA) bateria=1;
188    else if(voltaje>V_BAJA+HISTERESIS) bateria=0;
189    else if(bateria==2) bateria=1;
190  }
191  void publicar(){
192    if(ahora-pasoBLE<500) return;
```

SECCIÓN II / SOFTWARE · CINCO PRÁCTICAS

Código completo 04 · 5/5

Practica_04.ino · Líneas 193-235. Copiar los bloques en este orden en un solo archivo; los números impresos son solo una guía.

```
193    pasoBLE=ahora;
194    // <=20 bytes: compatible con ATT MTU 23. Sin datos privados ni audio.
195    char texto[21];
196    snprintf(texto,sizeof(texto),"%c E%d B%d Q%d",automatico?'A':'M',
197        evento,bateria,puntos);
198    estadoBLE->setValue(texto); estadoBLE->notify();
199    char v[12];
200    if(PILAS) snprintf(v,sizeof(v),"%.3f V",voltaje);
201    else snprintf(v,sizeof(v),"USB; NA");
202    voltajeBLE->setValue(v);
203  }
204
205  void setup(){
206    Serial.begin(115200); // sin esperar al monitor: funciona con pilas
207    pinMode(LED,OUTPUT); pinMode(BTN,INPUT_PULLUP);
208    iniciarIMU();
209    iniciarAudio();
210    iniciarBLE();
211    Serial.println("K9 CALM listo; boton cambia modo; l conmuta LED manual");
212  }
213  void loop(){
214    ahora=millis(); controles();
215    movimiento();
216    escuchar();
217    medirPilas();
218    if(impulso){ evento=1; alertaHasta=ahora+1000; }
219    publicar();
220    bool luz=automatico?((ahora/500)%2):manualLed;
221    if(automatico) luz=(actividad>0.7f);
222    if(automatico && int32_t(alertaHasta-ahora)>0) luz=true;
223    if(bateria) luz=((ahora/(bateria==1?500:125))%2);
224    digitalWrite(LED,luz);
225    if(ahora-informe>=250){
226      informe=ahora;
227      Serial.printf("modo:%d",automatico);
228      Serial.printf(" a:%.3f giro:%.3f act:%.3f",magnitud,giro,actividad);
229      Serial.printf(" rms:%.5f pico:%.5f ag:%.2f dur:%lu",rms,pico,agudos,
230          (unsigned long)duracion);
231      Serial.printf(" V:%.3f B:%d E:%d Q:%d",voltaje,bateria,evento,puntos);
232      Serial.println();
233    }
234    yield(); // cede CPU; no espera milisegundos fijos
235  }
```

Hecho por el profesor Francisco López Velázquez.
zolvek.com.mx

SECCIÓN II / SOFTWARE · CINCO PRÁCTICAS

05 / Integrar el detector

Objetivo. Relacionar un impulso con cambios de actividad y emitir una categoría experimental.

Lo que aprenderemos

Ventanas previas/posteriores, calidad de datos, estados, puntuación y enfriamiento.

Materiales y punto de partida

Todo H5 en USB, Android, multímetro y objeto/peluche; sin motor y sin animales.

Conservar H4 y agregar H5.

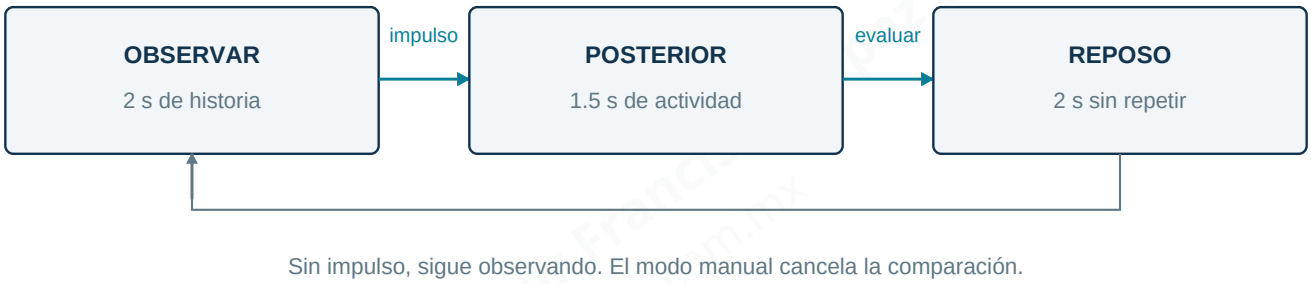


Figura P5. Secuencia de observacion y comparación.

Red	Conexión
Conservar	GPIO 10,3,4,5,6,7,1,0 según H1-H4
Energía	Sensores 3V3; USB o pilas validadas

SECCIÓN II / SOFTWARE · CINCO PRÁCTICAS

P5 · Decidir con memoria

No se añade una biblioteca. Abrir P5 completo: conserva controles, escáner I2C, lecturas, FFT, ADC condicional y BLE. En automático añade OBSERVAR, POSTERIOR y REPOSO. En manual mide y publica, pero desarma la clasificación; la letra l controla el LED, salvo que una alarma de alimentación tenga prioridad.

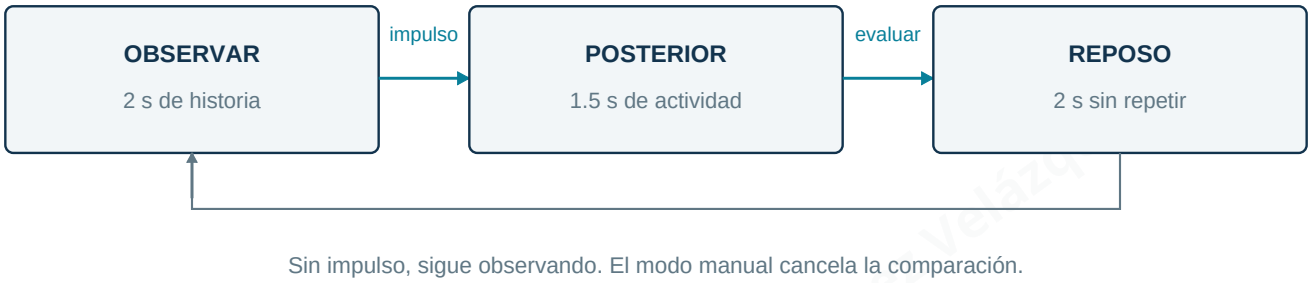


Figura 15. Transición por impulso, evaluacion tras 1.5 s y espera de 2 s antes de volver a observar.

OBSERVAR: historial circular de 100 valores de actividad, unos 2 s. Al detectar impulso se congela media previa y desviación estándar. **POSTERIOR:** acumula 1.5 s de actividad nueva. **REPOSO:** muestra la decisión y espera 2 s para evitar contar repetidamente el mismo episodio. La alerta visual dura 1.5 s, y BLE publica el estado durante el reposo.

La media previa resume el nivel habitual; sigma mide su variación. El cambio significativo exige **media posterior > media previa + max(0.7,3×sigma)**. Las unidades siguen siendo m/s^2. Si la actividad previa ya era alta o variable, sube el umbral. Este criterio puede perder una reacción breve; no es una regla universal de comportamiento.

Calidad suficiente requiere 100 lecturas previas, IMU reciente (<80 ms), audio reciente (<100 ms), pico inicial <0.98 y al menos 60 muestras posteriores. Un hueco durante la observacion posterior invalida la decisión. La comprobación temporal no detecta todos los errores de DMA ni verifica calibración física: inspeccionar también trazas y cables.

SECCIÓN II / SOFTWARE · CINCO PRÁCTICAS

P5 · Tabla de decisión y puntuación

Impulso	Datos suficientes	Cambio posterior	Categoría
No	Cualquiera	Cualquiera	E0 Actividad normal*
Si	No	Cualquiera	E3 Información insuficiente
Si	Si	No	E1 Evento acústico sospechoso
Si	Si	Si	E2 Posible reacción de sobresalto

*“Normal” significa sin evento pendiente según este detector; no certifica ausencia de miedo. Sin impulso, aun con movimiento, no se crea una reacción. E3 solo se produce después de un impulso recibido: un micrófono completamente desconectado puede impedirlo. Revisar sensores al inicio, no confiar solo en la categoría.

Puntuación: +60 por cambio de movimiento; +20 si ataque>0.01, de lo contrario +10; +20 si fraccion agudos>0.25. Si faltan datos, Q=0. Por ejemplo: cambio si, ataque 0.02 y agudos 0.30 produce Q=100. Sin cambio y con el mismo sonido, Q=40 pero sigue E1. La categoría depende de la tabla, no de un umbral secreto de Q.

La puntuación es una heurística para comparar ensayos, no probabilidad. Un sonido grave podría generar E2 y Q80, mientras uno agudo genera E1 y Q40. FFT y ataque describen el sonido; no prueban miedo ni identifican su fuente. Mantener distancia, volumen y fijación constantes para comparar resultados.

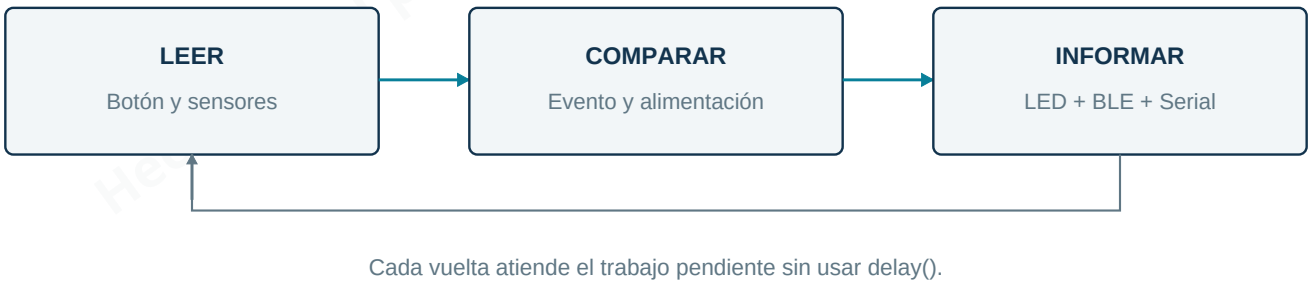


Figura 16. Orden cooperativo de tareas. No hay delay(); las operaciones I2C/FFT/ADC aun requieren tiempo finito.

Lectura por módulos

integrar() copia estadisticas previas al disparo, cuenta lecturas nuevas con ultimoIMU, valida calidad y decide una sola vez. **inicioEvento** fija la ventana posterior; **descanso** limita repeticiones. **alertaHasta** controla el LED. El aviso de batería tiene prioridad: lento para baja, rápido para crítica o ADC inválido. El programa no corta potencia.

SECCIÓN II / SOFTWARE · CINCO PRÁCTICAS

P5 · Protocolo guiado de integración

Encender y esperar 5 s con objeto quieto. Verificar 0x68, audio variable y BLE conectado. Trabajar con PILAS=false. Registrar fecha, firmware, distancia al altavoz, volumen, montaje y resultado; dejar al menos 5 s entre ensayos. Repetir cada condicion cinco veces.

Caso	Accion controlada	Esperado
1	Sin audio impulsivo, objeto quieto	E0; actividad baja
2	Mover objeto sin sonido	E0; actividad puede subir
3	Grabacion impulsiva moderada, sin mover	E1 si supera detector
4	Mismo sonido y mover suavemente el objeto durante 1.5 s	E2 si se supera cambio
5	Sonido antes de completar historia tras reinicio	E3 si hay impulso después del segundo inicial y antes de 2 s de historia
6	Modo manual; sonido y movimiento	M y E0; no clasifica
7	Dos impulsos separados menos de 2 s	Sin decisiones repetidas por el mismo episodio
8	Desconectar y reconectar BLE desde app	Vuelve a anunciar y permite consultar

Para probar E3 sin tocar cables energizados, reiniciar y emitir grabacion entre 1.2 y 1.8 s; si no dispara, repetir con calma. No retirar sensores con energía. Para simular falla del IMU, desconectar su alimentación sin USB y volver a encender; revisar mensaje de ausencia y una clasificación insuficiente si se detecta impulso.

Si no se obtiene el resultado previsto

Primero confirmar si hubo impulso: si rms no supera el umbral, no se abre ventana. Si E1 persiste al mover, revisar act y el tiempo del movimiento, sin aumentar volumen. Si E3, comprobar historia y nPost; no reducir controles de calidad para “forzar” E2. Si hay reinicios, detener ensayos y revisar alimentación antes de ajustar clasificadores.

Actividad de análisis y avance

Crear matriz de resultados de cinco repeticiones por caso. Calcular falsos positivos (E2 sin movimiento previsto) y falsos negativos (sin E2 cuando el ensayo lo esperaba). Son medidas del protocolo simulado, no precisión sobre perros. Proponer una mejora y justificarla con registros.

Verificación. Se conservan P1-P4 / E0/E1/E2/E3 explicados / Modo manual desarma / BLE informa / Sin repeticion inmediata / Sin uso animal

SECCIÓN II / SOFTWARE · CINCO PRÁCTICAS

Código completo 05 · 1/6

Practica_05.ino · Líneas 1-48. Copiar los bloques en este orden en un solo archivo; los números impresos son solo una guía.

```
// K9 CALM | Practica 5 | Arduino-ESP32 3.3.4
// LED externo activo en HIGH. No se usa LED integrado.
#include <Arduino.h>
#include <math.h>
constexpr int LED=10, BTN=3;
bool automatico=true, manualLed=false;
uint32_t ahora=0, cambio=0, informe=0;
bool crudo=HIGH, estable=HIGH;
int evento=0, puntos=0; // 0 normal, 1 sonido, 2 posible reaccion, 3 insuficiente
uint32_t alertaHasta=0;
void controles() {
  bool b=digitalRead(BTN);
  if(b!=crudo){ crudo=b; cambio=ahora; }
  if(ahora-cambio>=30 && b!=estable){
    estable=b;
    if(b==LOW){ automatico=!automatico; Serial.println(automatico?"AUTO":"MANUAL"); }
  }
  // En modo manual, enviar l (ele minuscula) conmuta el LED.
  if(Serial.available() && Serial.read()=='l') manualLed=!manualLed;
}

#include <Wire.h>
#include <Adafruit_MPU6050.h>
Adafruit_MPU6050 mpu;
bool imuOK=false;
float actividad=0, giro=0, magnitud=0;
float historial[100]={0}; // 2 s a 50 Hz; contiene actividad filtrada
int cabeza=0, cuantos=0;
uint32_t ultimoIMU=0, pasoIMU=0;
void iniciarIMU(){
  Wire.begin(4,5); Wire.setTimeOut(10); // SDA, SCL, limite de espera I2C
  Serial.println("Escaner I2C: se espera 0x68");
  for(int d=1;d<127;d++){
    Wire.beginTransmission(d);
    if(Wire.endTransmission()==0) Serial.printf("I2C 0x%02X\n",d);
  }
  imuOK=mpu.begin(0x68,&Wire); // AD0 a GND. Si falla, revisar y reiniciar.
  if(!imuOK){ Serial.println("MPU AUSENTE: revisar H2"); return; }
  mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
  mpu.setGyroRange(MPU6050_RANGE_500_DEG);
  mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);
}
void movimiento(){
  if(!imuOK || ahora-pasoIMU<20) return;
  pasoIMU=ahora;
  sensors_event_t a,g,t;
  if(!mpu.getEvent(&a,&g,&t)) return;
  // Unidades de Adafruit: m/s2 y rad/s (no grados/s).
```

SECCIÓN II / SOFTWARE · CINCO PRÁCTICAS

Código completo 05 · 2/6

Practica_05.ino · Líneas 49-96. Copiar los bloques en este orden en un solo archivo; los números impresos son solo una guía.

```
  magnitud=sqrtf(a.acceleration.x*a.acceleration.x+
      a.acceleration.y*a.acceleration.y+a.acceleration.z*a.acceleration.z);
  giro=sqrtf(g.gyro.x*g.gyro.x+g.gyro.y*g.gyro.y+g.gyro.z*g.gyro.z);
  float dinamica=fabsf(magnitud-9.80665f);
  actividad+=0.15f*(dinamica-actividad); // filtro exponencial
  historial[cabeza]=actividad; cabeza=(cabeza+1)%100;
  if(cuantos<100) cuantos++;
  ultimoIMU=ahora;
}

#include <driver/i2s_std.h>
i2s_chan_handle_t rx=nullptr;
bool audioOK=false, impulso=false;
constexpr int N=256, FS=16000;
float re[N], im[N], rms=0, pico=0, ruido=0.001f, agudos=0;
float ataque=0, decaimiento=0, rmsAnterior=0;
uint32_t ultimoAudio=0, inicioSonido=0, duracion=0, ultimoPulso=0;
bool sonando=false;
int llenos=0;
void iniciarAudio(){
  i2s_chan_config_t c=I2S_CHANNEL_DEFAULT_CONFIG(I2S_NUM_0,I2S_ROLE_MASTER);
  c.dma_desc_num=8; c.dma_frame_num=128;
  if(i2s_new_channel(&c,nullptr,&rx)!=ESP_OK) return;
  i2s_std_config_t s={};
  s.clk_cfg=I2S_STD_CLK_DEFAULT_CONFIG(FS);
  // Dos ranuras de 32 bits = 64 BCLK por trama. Solo usamos izquierda.
  s.slot_cfg=I2S_STD_PHILIPS_SLOT_DEFAULT_CONFIG(
      I2S_DATA_BIT_WIDTH_32BIT,I2S_SLOT_MODE_STEREO);
  s.gpio_cfg.mclk=I2S_GPIO_UNUSED; s.gpio_cfg.bclk=GPIO_NUM_6;
  s.gpio_cfg.ws=GPIO_NUM_7; s.gpio_cfg.dout=I2S_GPIO_UNUSED;
  s.gpio_cfg.din=GPIO_NUM_1;
  audioOK=(i2s_channel_init_std_mode(rx,&s)==ESP_OK &&
           i2s_channel_enable(rx)==ESP_OK);
  if(!audioOK) Serial.println("ERROR I2S: revisar version y pines");
}
// FFT radix-2 in situ. N debe ser potencia de 2.
void espectro(){
  for(int i=0;i<N;i++){
    re[i]*=0.5f-0.5f*cosf(2*PI*i/(N-1)); im[i]=0; // ventana Hann
  }
  for(int i=1,j=0;i<N;i++){
    int b=N>>1; for(;j&b;b>>=1) j^=b; j^=b;
    if(i<j){ float v=re[i]; re[i]=re[j]; re[j]=v; }
  }
  for(int len=2;len<=N;len<<=1){
    float ang=-2*PI/len;
    for(int i=0;i<N;i+=len){
      for(int j=0;j<len/2;j++){
```

SECCIÓN II / SOFTWARE · CINCO PRÁCTICAS

Código completo 05 · 3/6

Practica_05.ino · Líneas 97-144. Copiar los bloques en este orden en un solo archivo; los números impresos son solo una guía.

```
 97          int u=i+j,v=u+len/2;
 98          float c=cosf(ang*j),s=sinf(ang*j);
 99          float tr=c*re[v]-s*im[v],ti=s*re[v]+c*im[v];
100          re[v]=re[u]-tr; im[v]=im[u]-ti; re[u]+=tr; im[u]+=ti;
101        }
102      }
103    }
104    float total=0,alta=0;
105    for(int k=2;k<N/2;k++){
106      float e=re[k]*re[k]+im[k]*im[k]; total+=e;
107      if(k*FS/N>=2000) alta+=e;
108    }
109    agudos=total>1e-12f?alta/total:0; // proporcion, no identificador del sonido
110  }
111  void analizarAudio(){
112    float media=0, suma=0; pico=0;
113    for(int i=0;i<N;i++) media+=re[i]/N;
114    for(int i=0;i<N;i++){
115      re[i]-=media; suma+=re[i]*re[i]; pico=fmaxf(pico,fabsf(re[i]));
116    }
117    rms=sqrtf(suma/N);
118    ataque=fmaxf(0,rms-rmsAnterior); decaimiento=fmaxf(0,rmsAnterior-rms);
119    float umbral=fmaxf(0.004f,ruido*6);
120    bool fuerte=(rms>umbral);
121    // Calentamiento 1 s; no se aprende el ruido durante sonidos fuertes.
122    if(ahora<1000 || !fuerte) ruido+=0.01f*(rms-ruido);
123    impulso=(ahora>=1000 && fuerte && !sonando &&
124        pico/fmaxf(rms,1e-6f)>2.0f && ataque>ruido*2 &&
125        ahora-ultimoPulso>2000);
126    if(impulso) ultimoPulso=ahora;
127    if(fuerte && !sonando){ inicioSonido=ahora; sonando=true; }
128    if(sonando) duracion=ahora-inicioSonido;
129    if(!fuerte) sonando=false;
130    rmsAnterior=rms;
131    espectro(); // destruye re[]; se vuelve a llenar en la siguiente ventana
132    ultimoAudio=ahora;
133  }
134  void escuchar(){
135    impulso=false;
136    if(!audioOK) return;
137    int32_t par[64*2]; size_t leidos=0;
138    // Timeout 0: si DMA no tiene datos, volvemos sin esperar.
139    i2s_channel_read(rx,par,sizeof(par),&leidos,0);
140    for(size_t j=0;j+1<leidos/4;j+=2){
141      // 24 bits validos alineados a la izquierda; canal L/R conectado a GND.
142      re[llenos++]=(par[j]>>8)/8388608.0f;
143      if(llenos==N){ llenos=0; analizarAudio(); }
144    }
```

SECCIÓN II / SOFTWARE · CINCO PRÁCTICAS

Código completo 05 · 4/6

Practica_05.ino · Líneas 145-192. Copiar los bloques en este orden en un solo archivo; los números impresos son solo una guía.

```
145  }
146
147  #include <NimBLEDevice.h>
148  // Cambiar a true SOLO tras validar H4. En USB, desconectar el portapilas.
149  constexpr bool PILAS=false;
150  constexpr float CAL=1.000f, FACTOR=2.50f; // 150k arriba + 100k abajo
151  // Umbrales didacticos: sustituir por los determinados con carga y regulador real.
152  constexpr float V_BAJA=4.00f, V_CRITICA=3.80f, HISTERESIS=0.15f;
153  float voltaje=0; int bateria=0; // 0 USB/normal, 1 baja, 2 critica, 3 ADC invalido
154  uint32_t pasoADC=0, pasoBLE=0;
155  NimBLECharacteristic *estadoBLE=nullptr, *voltajeBLE=nullptr;
156  const char* SERVICIO="8d6c0001-3d68-4b85-9b71-4d3943414c4d";
157  class Conexion:public NimBLEServerCallbacks{
158    void onDisconnect(NimBLEServer*,NimBLEConnInfo&,int) override{
159      NimBLEDevice::startAdvertising();
160    }
161  } conexion;
162  void iniciarBLE(){
163    NimBLEDevice::init("K9-CALM");
164    auto s=NimBLEDevice::createServer(); s->setCallbacks(&conexion,false);
165    auto servicio=s->createService(SERVICIO);
166    estadoBLE=servicio->createCharacteristic(
167        "8d6c0002-3d68-4b85-9b71-4d3943414c4d",
168        NIMBLE_PROPERTY::READ|NIMBLE_PROPERTY::NOTIFY);
169    voltajeBLE=servicio->createCharacteristic(
170        "8d6c0003-3d68-4b85-9b71-4d3943414c4d",NIMBLE_PROPERTY::READ);
171    servicio->start();
172    auto a=NimBLEDevice::getAdvertising();
173    a->setName("K9-CALM"); a->addServiceUUID(SERVICIO);
174    a->enableScanResponse(true); a->start();
175    analogReadResolution(12); analogSetPinAttenuation(0,ADC_11db);
176  }
177  void medirPilas(){
178    if(!PILAS || ahora-pasoADC<1000) return;
179    pasoADC=ahora;
180    uint32_t suma=0;
181    for(int j=0;j<16;j++) suma+=analogReadMilliVolts(0);
182    float mv=suma/16.0f; voltaje=mv*FACTOR*CAL/1000;
183    if(mv<100 || mv>2400){ bateria=3; return; }
184    if(bateria==3) bateria=0;
185    if(voltaje<V_CRITICA) bateria=2;
186    else if(bateria==2 && voltaje<V_CRITICA+HISTERESIS) return;
187    else if(voltaje<V_BAJA) bateria=1;
188    else if(voltaje>V_BAJA+HISTERESIS) bateria=0;
189    else if(bateria==2) bateria=1;
190  }
191  void publicar(){
192    if(ahora-pasoBLE<500) return;
```

SECCIÓN II / SOFTWARE · CINCO PRÁCTICAS

Código completo 05 · 5/6

Practica_05.ino · Líneas 193-240. Copiar los bloques en este orden en un solo archivo; los números impresos son solo una guía.

```
193    pasoBLE=ahora;
194    // <=20 bytes: compatible con ATT MTU 23. Sin datos privados ni audio.
195    char texto[21];
196    snprintf(texto,sizeof(texto),"%c E%d B%d Q%d",automatico?'A':'M',
197        evento,bateria,puntos);
198    estadoBLE->setValue(texto); estadoBLE->notify();
199    char v[12];
200    if(PILAS) snprintf(v,sizeof(v),"%.3f V",voltaje);
201    else snprintf(v,sizeof(v),"USB; NA");
202    voltajeBLE->setValue(v);
203  }
204
205  enum Fase{OBSERVAR,POSTERIOR,REPOSO};
206  Fase fase=OBSERVAR;
207  float previa=0, sigma=0, sumaPost=0, ataqueEvento=0, agudosEvento=0;
208  int nPost=0; bool suficiente=false;
209  uint32_t inicioEvento=0, descanso=0, contadoIMU=0;
210  void integrar(){
211    // Modo manual conserva medidas y BLE; desarma la clasificacion automatica.
212    if(!automatico){ fase=OBSERVAR; evento=0; puntos=0; return; }
213    if(fase==REPOSO){
214      if(ahora-descanso>=2000){fase=OBSERVAR; evento=0; puntos=0;}
215      return;
216    }
217    if(fase==OBSERVAR && impulso){
218      previa=0; sigma=0;
219      for(int j=0;j<cuantos;j++) previa+=historial[j]/cuantos;
220      for(int j=0;j<cuantos;j++) sigma+=sq(historial[j]-previa)/cuantos;
221      sigma=sqrtf(sigma);
222      suficiente=(cuantos==100 && ahora-ultimoIMU<80 &&
223                  ahora-ultimoAudio<100 && pico<0.98f);
224      ataqueEvento=ataque; agudosEvento=agudos;
225      sumaPost=0; nPost=0; inicioEvento=ahora; contadoIMU=ultimoIMU;
226      evento=1; fase=POSTERIOR;
227    }
228    if(fase!=POSTERIOR) return;
229    if(ahora-ultimoIMU>80 || ahora-ultimoAudio>100) suficiente=false;
230    if(ultimoIMU!=contadoIMU){
231      contadoIMU=ultimoIMU; sumaPost+=actividad; nPost++;
232    }
233    if(ahora-inicioEvento<1500) return;
234    float posterior=nPost?sumaPost/nPost:0;
235    bool cambioMov=(posterior>previa+fmaxf(0.7f,3*sigma));
236    // Puntuacion exploratoria; no es probabilidad ni diagnostico.
237    puntos=(cambioMov?60:0)+(ataqueEvento>0.01f?20:10)+
238           (agudosEvento>0.25f?20:0);
239    suficiente=suficiente && nPost>=60;
240    evento=!suficiente?3:(cambioMov?2:1);
```

SECCIÓN II / SOFTWARE · CINCO PRÁCTICAS

Código completo 05 · 6/6

Practica_05.ino · Líneas 241-277. Copiar los bloques en este orden en un solo archivo; los números impresos son solo una guía.

```
241    if(!suficiente) puntos=0;
242    alertaHasta=ahora+1500; descanso=ahora; fase=REPOSO;
243    Serial.printf("Evento=%d Q=%d pre=%.2f post=%.2f n=%d\n",
244        evento,puntos,previa,posterior,nPost);
245  }
246
247  void setup(){
248    Serial.begin(115200); // sin esperar al monitor: funciona con pilas
249    pinMode(LED,OUTPUT); pinMode(BTN,INPUT_PULLUP);
250    iniciarIMU();
251    iniciarAudio();
252    iniciarBLE();
253    Serial.println("K9 CALM listo; boton cambia modo; l conmuta LED manual");
254  }
255  void loop(){
256    ahora=millis(); controles();
257    movimiento();
258    escuchar();
259    medirPilas();
260    integrar();
261    publicar();
262    bool luz=automatico?((ahora/500)%2):manualLed;
263    if(automatico) luz=(actividad>0.7f);
264    if(automatico && int32_t(alertaHasta-ahora)>0) luz=true;
265    if(bateria) luz=((ahora/(bateria==1?500:125))%2);
266    digitalWrite(LED,luz);
267    if(ahora-informe>=250){
268      informe=ahora;
269      Serial.printf("modo:%d",automatico);
270      Serial.printf(" a:%.3f giro:%.3f act:%.3f",magnitud,giro,actividad);
271      Serial.printf(" rms:%.5f pico:%.5f ag:%.2f dur:%lu",rms,pico,agudos,
272        (unsigned long)duracion);
273      Serial.printf(" V:%.3f B:%d E:%d Q:%d",voltaje,bateria,evento,puntos);
274      Serial.println();
275    }
276    yield(); // cede CPU; no espera milisegundos fijos
277  }
```

Hecho por el profesor Francisco López Velázquez. zolvek.com.mx

CIERRE / VERIFICACIÓN Y CONSULTA

Pruebas finales y límites

Comprobación	Criterio / evidencia a registrar
Inspeccion sin energía	Continuidad de cada red; sin corto 3V3-GND; serigrafía coincide.
USB solamente	Cinco prácticas arrancan sin pack; monitor no es necesario para operar.
Sensores	0x68 estable; a cercana a gravedad en reposo; rms dinamico sin saturación.
Carga de trabajo	Con BLE suscrito, lecturas recientes y nPost >=60. Sin reinicios 10 minutos de banco.
Pilas (condicional)	Ficha LDO, barrido bajo carga, umbrales ajustados y temperaturas registradas.
Mecánica	Caja aislada, conector desmontable, sensor firme, mic y antena libres.
Documentación	Registro distingue ensayo sintético, compilación y medida física.

Registro breve por equipo

Placa/marcaje LDO: _____ Firmware: _____
3V3 con USB: _____ V Entrada mínima estable: _____ V
Corriente promedio/pico observada: _____ Instrumento: _____
CAL: _____ Baja/crítica validadas: _____ / _____ V
Resultados casos P5: _____
Responsable y fecha: _____

Alcance. No hay hardware físico conectado en esta elaboracion. Las conexiones se revisan teoricamente; los ensayos reales de alimentación, radio, sonido y bienestar quedan pendientes. La compilación registrada no certifica un montaje.

Para usar una pechera fuera del banco se requieren evaluaciones de seguridad eléctrica, térmica y mecánica, y validación profesional de bienestar. El motor sigue siendo una extensión de laboratorio. Clasificar eventos no constituye tratamiento del miedo.

CIERRE / VERIFICACIÓN Y CONSULTA

Diagnosticar por síntomas · 1

Síntoma	Posible causa	Como comprobar	Solucion
PC no detecta placa	Cable solo carga o USB sin datos	Cambiar cable/puerto; comparar listado antes/después	Cable de datos; identificar USB nativo o puente
Error de compilación	Placa/versión equivocada o sintaxis	Leer primer error y core seleccionado	ESP32C3 Dev Module; core 3.3.4; código completo
No encuentra .h	Biblioteca faltante	Gestor de bibliotecas y ruta del error	Instalar versiones fijadas y dependencias
Falla al subir	Puerto ocupado o arranque	Cerrar monitor/otras apps; revisar nuevo puerto	BOOT+RESET; elegir puerto de arranque; 115200
Serial ilegible/vacio	Baudrate o CDC	Monitor a 115200; CDC Enabled	Corregir opciones, subir, RESET
Error en callback NimBLE	Otra versión de biblioteca	Leer versión y firma onDisconnect	NimBLE-Arduino 2.5.2; quitar duplicados
MPU no detectado	SDA/SCL, AD0 o alimentación	Escáner; medir reposo líneas y VCC	4 SDA, 5 SCL, AD0 GND; pull-ups 3V3
Movimiento absurdo	Módulo suelto o rango saturado	Fijar en mesa; revisar norma y orientación	Sujetar, comprobar VCC/rango, repetir
Sin audio	L/R equivocado o SD incorrecto	Ver H3 y RMS, inspeccionar reloj si posible	L/R GND; SD 1; SCK6; WS7
Muestras I2S incorrectas	Ranuras/formato inadecuados	Revisar 32 bits estereo y desplazamiento	Usar P3 fijado; no mezclar API legacy
Pico saturado	Audio muy fuerte o formato erroneo	Pico ~ 1 aun en silencio; bajar volumen	Corregir formato, desacoplo y distancia
GPIO no responde	Se confundio posición con número	Continuidad sin energía y serigrafía	Seguir mapa; GPIO10 no es pin físico 10

CIERRE / VERIFICACIÓN Y CONSULTA

Diagnosticar por síntomas · 2

Síntoma	Posible causa	Como comprobar	Solucion
Reinicios	Caída en LDO, cable o sobrecorriente	Leer brownout; medir entrada/3V3 bajo BLE	Volver USB, validar H4; no desactivar brownout
Voltaje incorrecto	R1/R2 invertidas o CAL mal	Medir resistencias sin energía y nodo con CC	150k arriba/100k abajo; recalibrar
ADC inestable	Nodo alto en impedancia o GND flojo	Comparar multímetro en tres voltajes	100nF cerca GPIO0; promedio; soldar GND
B3 permanente	Entrada ADC fuera de 100-2400mV	Medir nodo, revisar PILAS y fuente	Reparar divisor; en USB dejar PILAS=false
Crítica muy temprano	Dropout o umbral sin validar	Barrido H4 y tensión después D1	Fijar umbrales con margen; rechazar AAA si no estable
BLE no aparece	Firmware no P4/P5 o permisos	Serial; Scanner; nombre y UUID	Conceder permisos; reiniciar; desconectar otro cliente
No hay notificaciones	No suscrito o app desconectada	Leer característica y activar suscripción	Suscribirse a 0002; no esperar aviso del SO
LED no enciende	Polaridad, resistencia, modo o B	Modo manual + l; medir GPIO10	Corregir H1; revisar alarma con prioridad
Falsos positivos	Ruido, cables sueltos o actividad previa	Repetir control sin sonido y sin mover	Fijar montaje; calibrar umbral con registros
P5 siempre E1	No cambia media posterior	Leer pre/post y sincronizar movimiento	Mover después del impulso; revisar metrica
P5 siempre E3	Historia corta o huecos de lectura	Esperar 5s; revisar nPost y mensajes	Corregir IMU/audio; quitar tareas largas
P5 deja de actualizar	Bloqueos añadidos o biblioteca diferente	Probar P3/P4; comparar fuentes	Restaurar versiones y loop sin delay
No se recupera tras reparar	Inicializacion solo ocurre en setup	Comprobar mensaje MPU AUSENTE	Apagar, corregir cableado y reiniciar

CIERRE / VERIFICACIÓN Y CONSULTA

Glosario de bolsillo

Término	Significado en este proyecto
ADC	Convertidor de voltaje a un valor digital. No se usa para el micrófono I2S.
Amplitud / pico	Valor de una muestra / mayor valor absoluto en una ventana.
Antirrebote	Acepta el botón solo tras 30 ms de estabilidad.
BLE / GATT	Enlace de bajo consumo / organizacion en servicios y características.
Brownout / dropout	Reinicio por alimentación baja / margen mínimo que necesita el regulador.
Calibración	Comparación con una referencia para reducir error; no crea exactitud ilimitada.
DMA	Hardware que transfiere muestras a memoria mientras CPU hace otras tareas.
FFT / bin	Transformada rápida de Fourier / intervalo de frecuencia del espectro.
GPIO / GND	Pin de entrada/salida / referencia común de potencial.
Gravedad / g	Aceleración de referencia; g ~ 9.80665 m/s^2. No confundir con gramos.
Histéresis	Umbrales distintos para entrar y salir de una alarma.
I2C / SDA / SCL	Bus de registros / datos / reloj. Dirección MPU: 0x68.
I2S / BCLK / WS / SD	Bus de audio / reloj de bits / ranura / datos digitales.
LDO	Regulador lineal de baja caída; en la placa produce 3.3 V.
Muestreo / Hz	Tomar valores periódicamente / ciclos o muestras por segundo según contexto.
Pull-up / pull-down	Resistencia que define alto / bajo cuando nadie conduce la línea.
RMS / Hann	Medida cuadratica de amplitud / ventana para reducir fuga espectral.
Sigma / norma	Desviación estándar / longitud del vector de tres ejes.
UUID / MTU	Identificador de servicio o dato / tamaño máximo de paquete negociado.
Velocidad angular	Rapidez de giro; rad/s en las lecturas usadas aqui.

CIERRE / VERIFICACIÓN Y CONSULTA

Fuentes técnicas y licencias

Consulta documental: 27 de octubre de 2026. Se revisaron README, licencias y actividad de los seis repositorios solicitados, más Adafruit Unified Sensor. Las fuentes se aplicaron al formato I2S, API del sensor, callbacks BLE y selección del dibujo por terminales. Versiones fijadas, no equivalentes a “última versión”.

R1 · ESP32-C3, Espressif

https://www.espressif.com/sites/default/files/documentation/esp32-c3_datasheet_en.pdf

Datos eléctricos, GPIO, strapping y radio. Complementar con guía de hardware.

R2 · Arduino-ESP32

https://github.com/espressif/arduino-esp32

Core 3.3.4; LGPL-2.1. Se usan API GPIO/ADC y driver estándar incluido.

R3 · Instalación oficial

https://docs.espressif.com/projects/arduino-esp32/en/latest/installing.html

Gestor de placas y URL oficial.

R4 · ADC oficial

https://docs.espressif.com/projects/arduino-esp32/en/latest/api/adc.html

analogReadMilliVolts, resolución y rangos por chip.

R5 · I2S ESP-IDF

https://docs.espressif.com/projects/esp-idf/en/v5.5/esp32c3/api-reference/peripherals/i2s.html

Controlador std, ranuras Philips, DMA y lectura con timeout.

R6 · INMP441, InvenSense

https://www.invensense.com/wp-content/uploads/2015/02/INMP441.pdf

Formato de 24 bits, 64 relojes, L/R y pulldown SD 100k.

Hecho por el profesor Francisco López Velázquez. zolvek.com.mx

CIERRE / VERIFICACIÓN Y CONSULTA

Bibliotecas y versiones fijadas

R7 · Adafruit MPU6050

https://github.com/adafruit/Adafruit_MPU6050

2.2.9; BSD-3-Clause (license.txt). API getEvent, rangos y unidades.

R8 · Adafruit BusIO

https://github.com/adafruit/Adafruit_BusIO

1.17.4; MIT. Dependencia de comunicación.

R9 · Unified Sensor

https://github.com/adafruit/Adafruit_Sensor

1.1.15; Apache-2.0. Estructuras de lecturas.

R10 · NimBLE-Arduino

https://github.com/h2zero/NimBLE-Arduino

2.5.2; Apache-2.0 y avisos de terceros. Servidor/advertising/callbacks.

Hecho por el profesor Francisco López Velázquez. zolvek.com.mx

CIERRE / VERIFICACIÓN Y CONSULTA

Recursos visuales y reproducibilidad

R11 · Fritzing

https://github.com/fritzing/fritzing-app

Aplicacion revisada; código GPLv3 y otras licencias por componente. No instalada; no se ejecutaron scripts ni se distribuye la aplicacion.

R12 · Adafruit Fritzing Library

https://github.com/adafruit/Fritzing-Library

CC BY-SA 3.0 según LICENSE. Revisada como referencia de representación; no se copiaron piezas gráficas ni imagenes.

R13 · SuperMini, esquema de distribuidor

https://dl.artronshop.co.th/ESP32-C3%20SuperMini%20datasheet.pdf

Referencia secundaria para columnas de 16 pines. No identifica de forma universal el regulador ni reemplaza el esquema de la unidad real.

R14 · MPU6050, guía Adafruit

https://learn.adafruit.com/mpu6050-6-dof-accelerometer-and-gyro

Explicacion de aceleración/giro y uso de biblioteca; no asumir que breakout Adafruit y GY-521 sean electricamente idénticos.

R15 · Nordic nRF Connect

https://www.nordicsemi.com/Products/Development-tools/nRF-Connect-for-mobile

App BLE Android; controles pueden cambiar entre versiones.

Hecho por el profesor Francisco López Velázquez.
zolvek.com.mx

CIERRE / VERIFICACIÓN Y CONSULTA

Herramientas de esta edición

R16 · Arduino IDE

https://www.arduino.cc/en/software

Descarga del entorno; usar distribucion oficial.

R17 · Guía hardware C3

https://docs.espressif.com/projects/esp-hardware-design-guidelines/en/latest/esp32c3/schematic-checklist.html

Recomendación de fuente y desacoplo del chip.

Los esquemas eléctricos se dibujan en SVG y conservan terminales verificables. La portada utiliza una ilustracion conceptual generada con IA, identificada como tal. Las vistas de placa y sensores son esquematizaciones identificadas; el orden físico de sensores varia por proveedor. Las gráficas son sintéticas. Se distribuyen SVG editables y PNG de alta resolución.

Herramientas usadas: skill pdf, Python/ReportLab, PyMuPDF, Pillow y Poppler. Fritzing y KiCad CLI no estaban disponibles; los diagramas SVG por terminal ofrecen una alternativa auditable. No se encontro una skill especializada en Arduino entre las disponibles; no se invento una. La generacion de imagen se uso para la portada conceptual; no para inventar pines o cableado.

Hecho por el profesor Francisco López Velázquez.
zolvek.com.mx

CIERRE / VERIFICACIÓN Y CONSULTA

Registro de verificación de esta entrega

Práctica	Verificación de compilación
1	Compilada correctamente
2	Compilada correctamente
3	Compilada correctamente
4	Compilada correctamente
5	Compilada correctamente

Configuración de referencia: Arduino-ESP32 3.3.4, ESP32C3 Dev Module (esp32:esp32:esp32c3), USB CDC habilitado; Adafruit MPU6050 2.2.9, BusIO 1.17.4, Unified Sensor 1.1.15 y NimBLE-Arduino 2.5.2. Las cinco fuentes completas se imprimen en el manual y se entregan como proyectos separados.

Nivel	Resultado / límite
Revisión eléctrica teórica	Sin conflicto entre GPIO asignados; USB18/19 y strapping2/8/9 reservados. Sensores a 3.3 V; divisor 150k/100k y fuente única.
Cálculos	R LED ~ 4mA; divisor×2.50; I2S16k×64=1.024MHz; FFT62.5Hz/bin; historial2s.
Pruebas numéricas	FFT con tonos 500/4000 Hz y silencio; E1/E2/E3, reposo, modo manual e histéresis: verificadas con funciones extraídas del firmware. No emulan periféricos.
Ejecución física	No realizada: faltan placa real, instrumentos, carga, sensores y ensayo de AAA.
Clasificador	Heurística experimental; faltan datos etiquetados y validación sobre condiciones reales.
Portátil	No aprobada por este documento: depende de LDO identificado y mediciones H4.

La carpeta de fuentes incluye los registros de compilación, bibliotecas/versiones, diagramas originales y generador. El README explica como regenerar sin editar directamente el PDF. El paquete ZIP conserva la estructura K9_Calm solicitada; en el repositorio el PDF final reside exclusivamente en output/pdf/K9_Calm/.